



Programming Multi-/Many-cores Using OpenCL

(CS 3006)

Adapted from: see the Acknowledgement slide
Compiled by: Dr. Muhammad Arshad Islam and Dr. Muhammad Aleem

National University of Computer & Emerging Sciences,
Islamabad Campus



Lecture

Acknowledgements

Simon McIntosh-Smith University of Bristol

http://people.cs.bris.ac.uk/~simonm/SC13/OpenCL_slides_SC13.pdf

Romain Teissier, Pierre-François Lavallée, et al.

http://irfu.cea.fr/Phocea/Vie_des_labos/Ast/ast_sstechnique.php?id_ast=904

Optimizing OpenCL applications on Intel Xeon Phi

<http://iwocl.org/wp-content/uploads/2013/06/Optimizing-OpenCL-Applications-on-Intel-Xeon-Phi-IWOCL.pdf>

OpenCL home page

<http://www.khronos.org/opencl/>

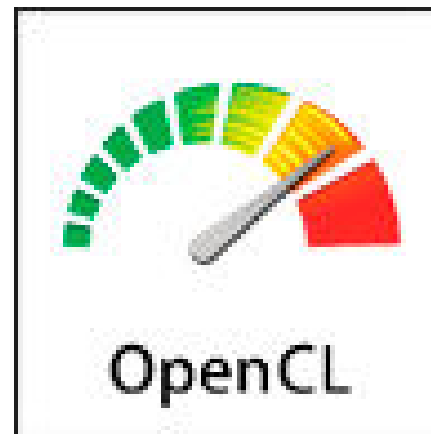
https://www.khronos.org/assets/uploads/developers/library/2012-pan-pacific-road-show-June/OpenCL-Details-Taiwan_June-2012.pdf

AMD

https://indico.fysik.su.se/event/1621/sessions/70/attachments/600/695/OpenCL_Training.pdf

OpenCL

- **Open Compute Language**
- For heterogeneous parallel-computing systems
- Cross-platform
 - Implementations for
 - ATI GPUs
 - NVIDIA GPUs
 - Intel MIC
 - x86 CPUs
 - Many others.....





OpenCL is Widely Deployed and Used



KHRONOS
GROUP

Non-profit Standards Consortium
creating open, royalty-free standards

Focused on runtime APIs and file formats
for 3D, XR, AI, vision, and
parallel compute acceleration

Member-driven, open to any company

Founded in 2000

~ 160 Members | ~ 40% US, 30% Europe, 30% Asia

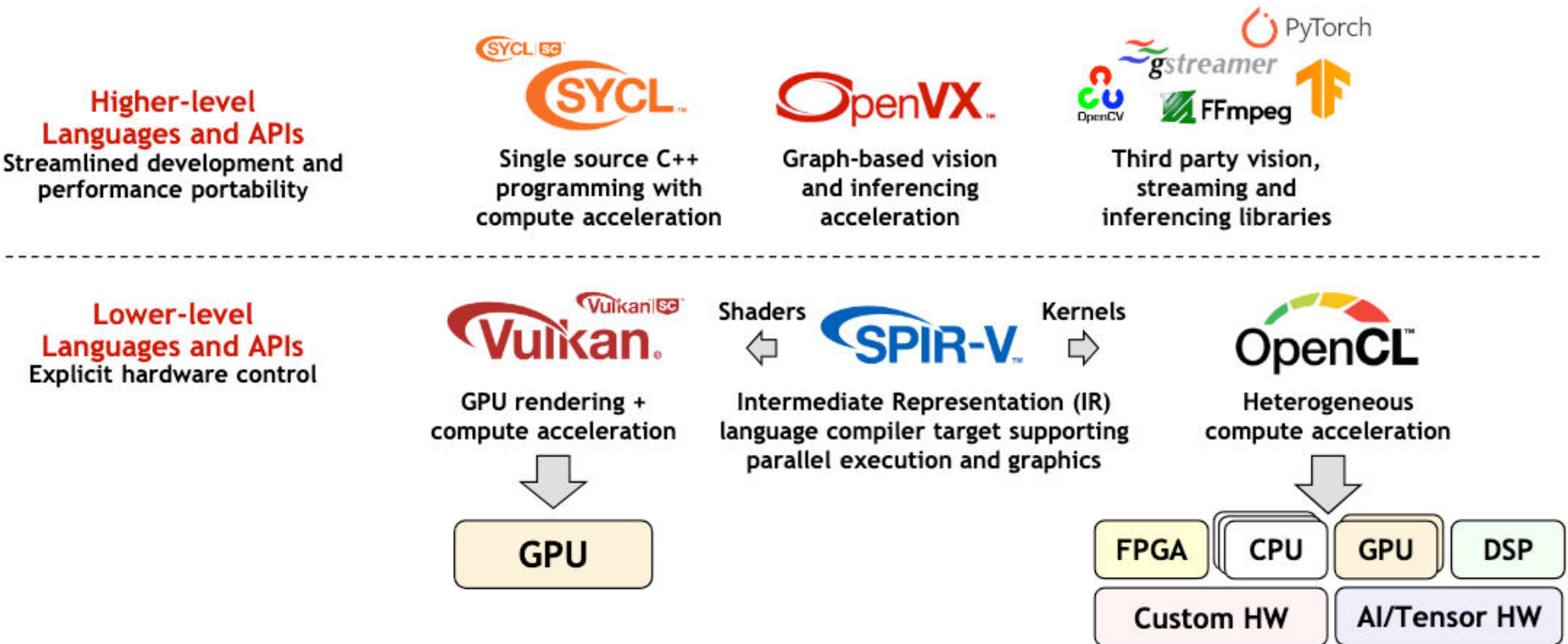
ISO/IEC JTC 1 PAS Submitter



Khronos Compute Acceleration

Choice of programming models to meet the needs of diverse developers

Higher-level applications, libraries, and languages often access hardware acceleration through lower-level APIs





Industry Standards for Programming Heterogeneous Platforms

Programming and Runtime Framework for Application Acceleration

Offload compute-intensive kernels onto parallel heterogeneous processors
CPUs, GPUs, DSPs, FPGAs, Tensor Processors
OpenCL C or C++ kernel languages

Platform Layer API

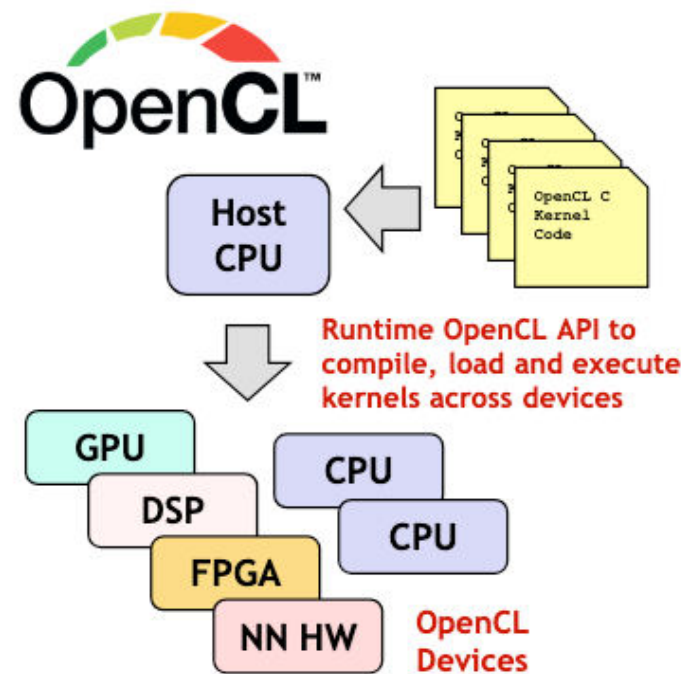
Query, select and initialize compute devices

Runtime API

Build and execute kernels programs on multiple devices

Explicit Application Control

Which programs execute on what device
Where data is stored in memories in the system
When programs are run, and what operations are dependent on earlier operations



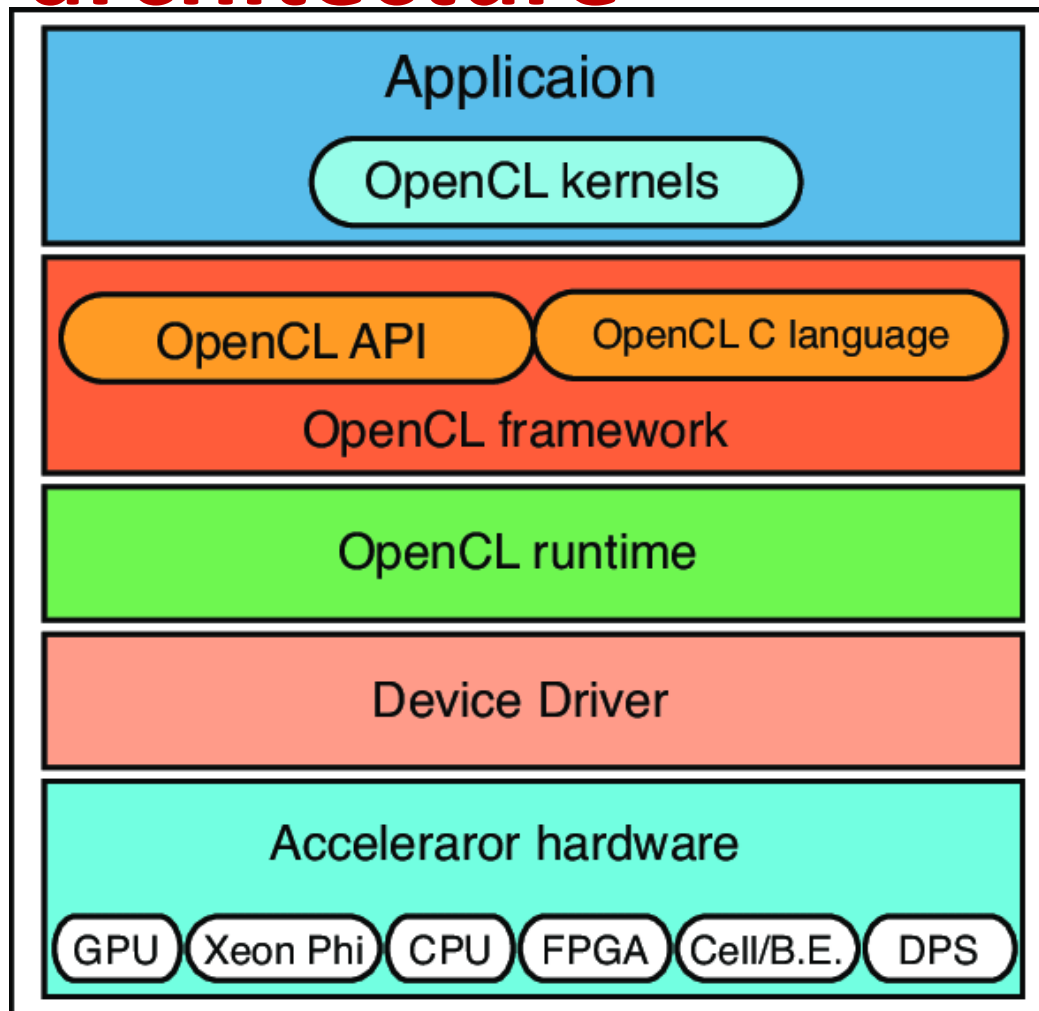
Complements GPU-only APIs

Simpler programming model
Relatively lightweight run-time
More language flexibility, e.g., pointers
Rigorously defined numeric precision

Open, royalty-free standard for portable, parallel programming of heterogeneous parallel computing CPUs, GPUs, and other processors

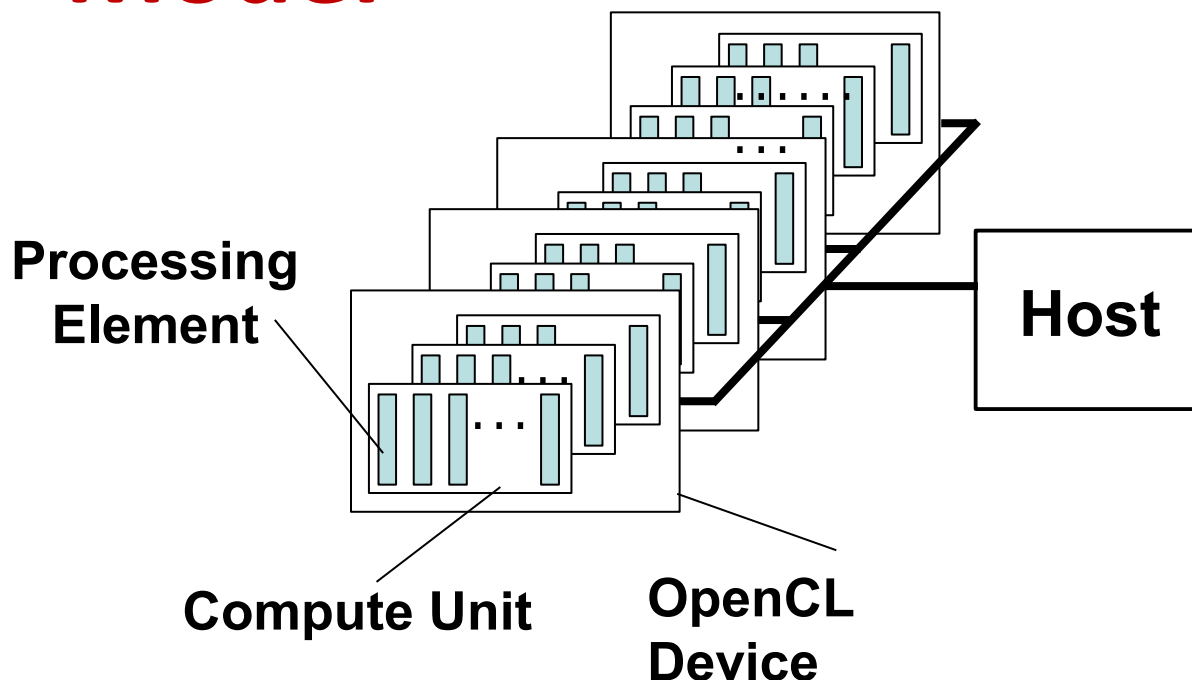


OpenCL architecture





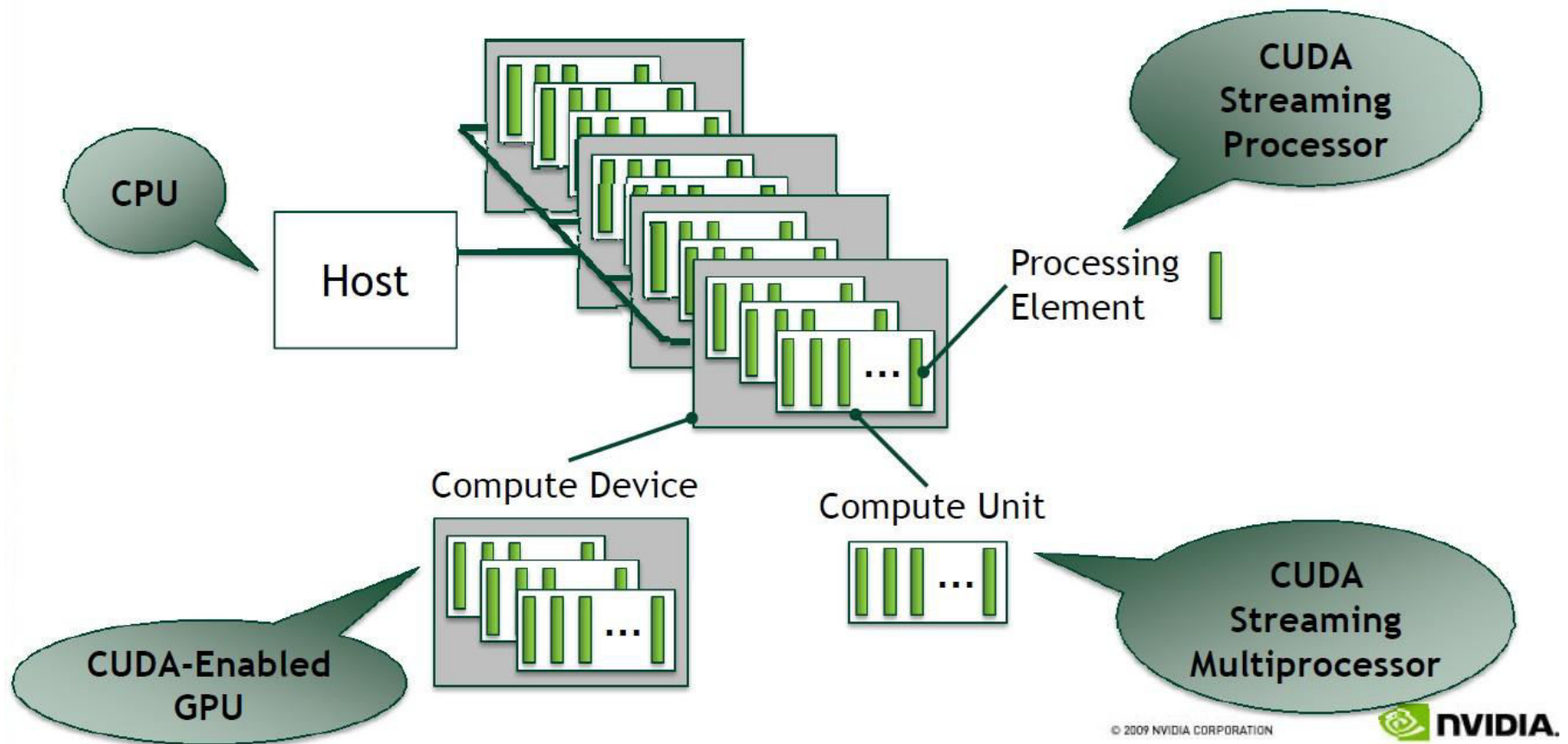
OpenCL Platform Model



- One **Host** and one or more **OpenCL Devices**
 - Each OpenCL Device is composed of one or more **Compute Units**
 - Each Compute Unit is divided into one or more **Processing Elements**
- Memory divided into **host memory** and **device memory**



OpenCL Platform Model on the CUDA Architecture

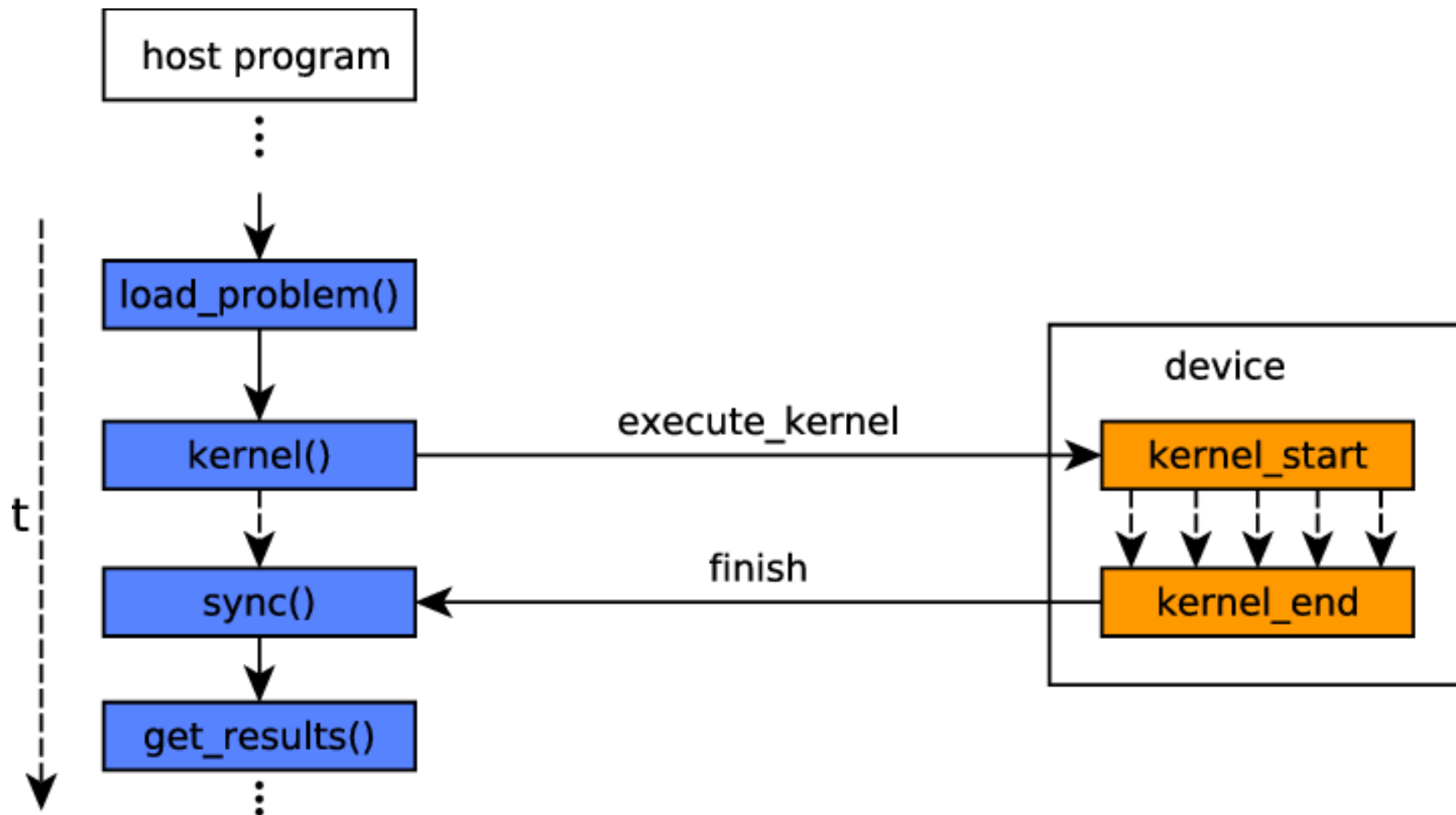


© 2009 NVIDIA CORPORATION





OpenCL Program: Host and Kernel





OpenCL Platform

(One node, two CPU sockets, two GPUs)

Example

CPU:

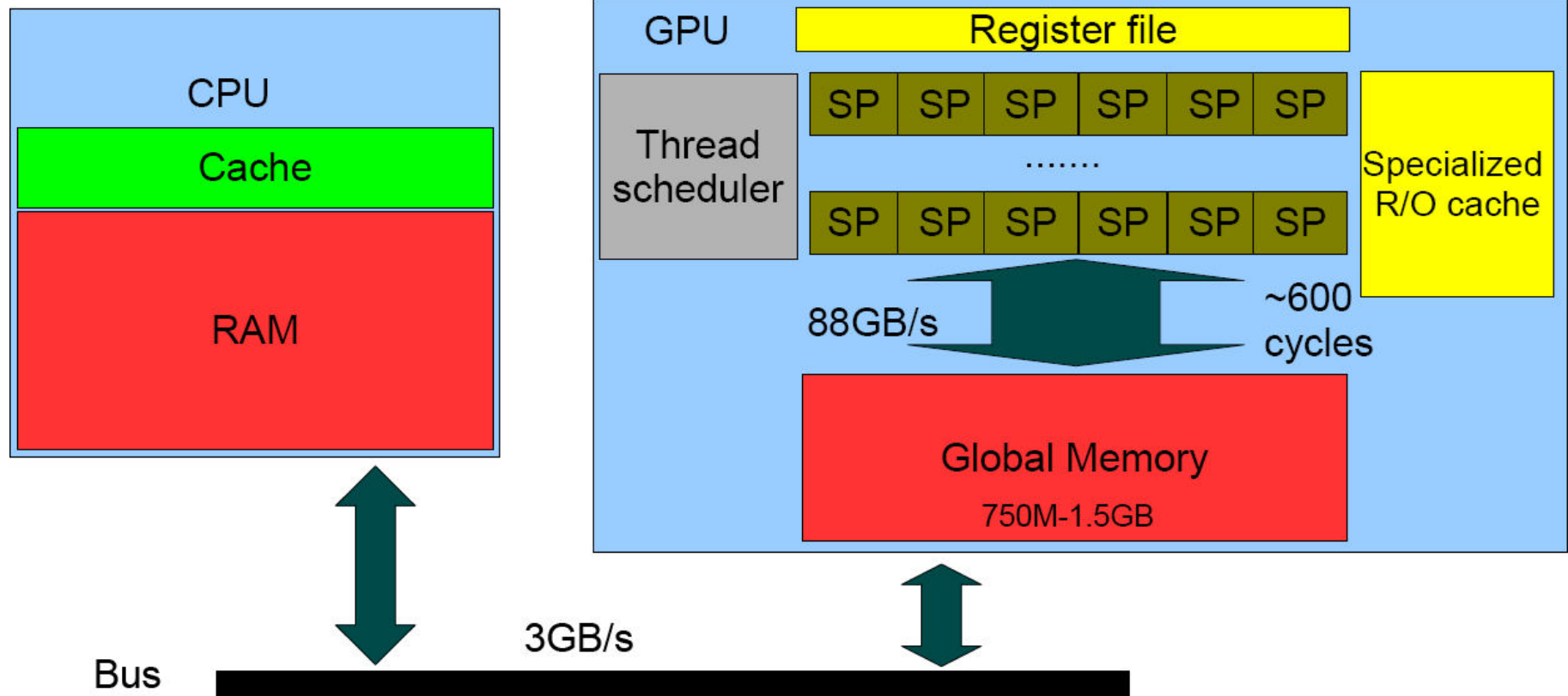
- Treated as one OpenCL device
 - One CU per core
 - 1 PE per CU, or if PEs mapped to SIMD lanes, n PEs per CU, where n matches the SIMD width
- Remember:
 - the CPU will also have to be its own host!

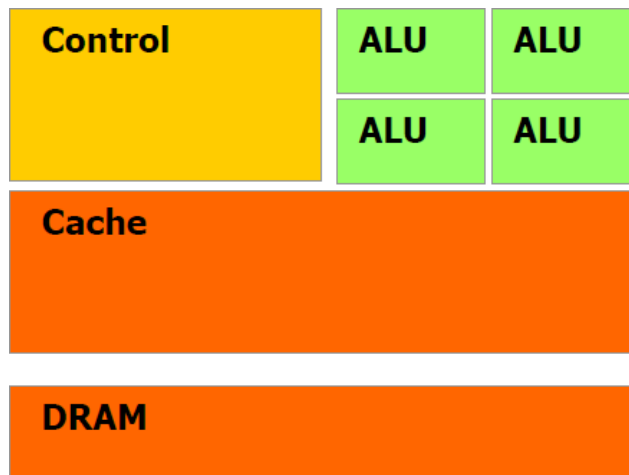
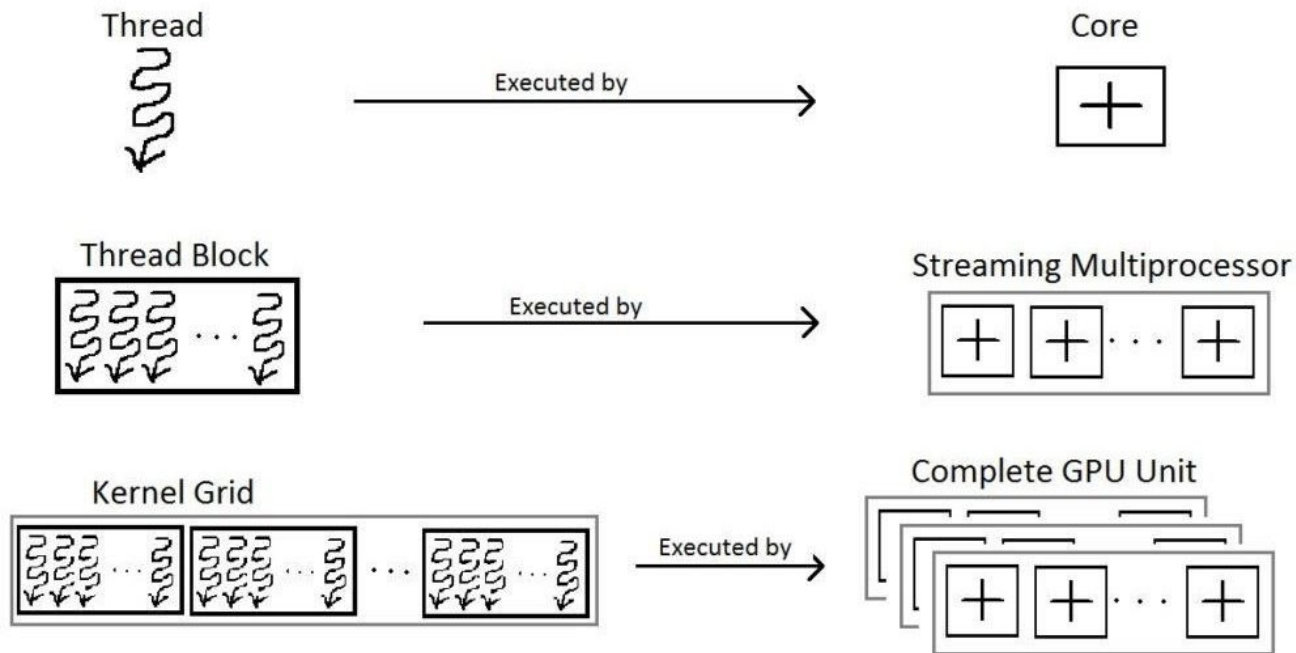
GPUs:

- Each GPU is a separate OpenCL device
- One CU per Streaming Multiprocessor
- Can use CPU and all GPU devices concurrently through OpenCL

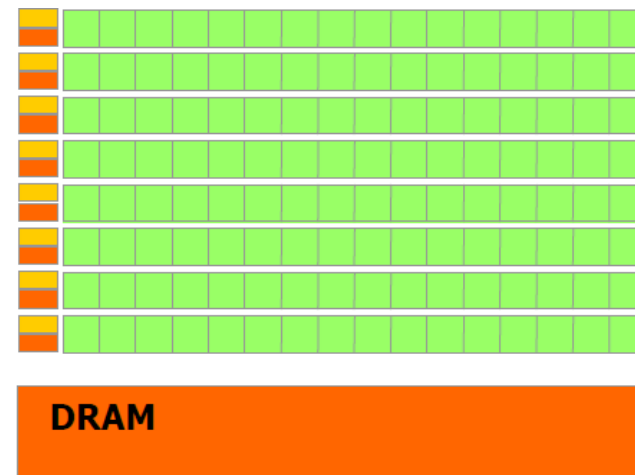
CU = Compute Unit; PE = Processing Element

Heterogeneous Computing – A High level View





CPU

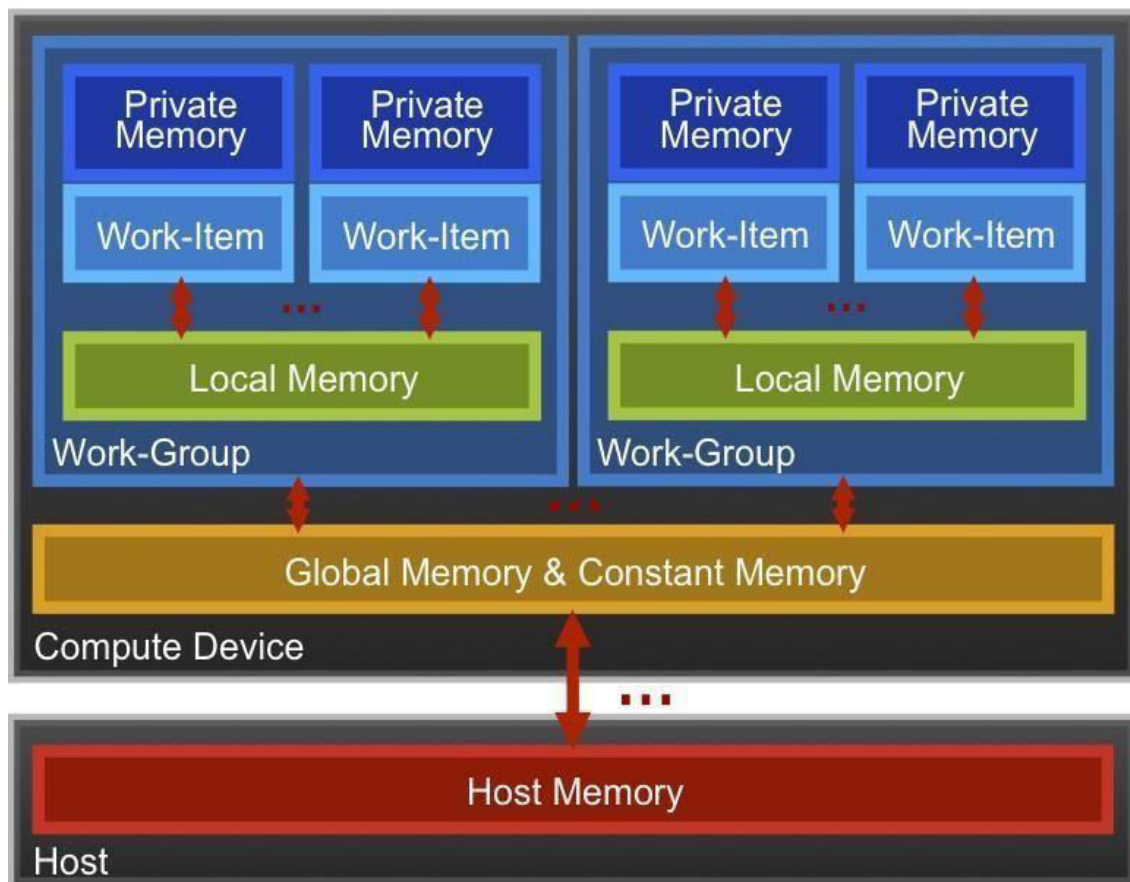


GPU



OpenCL Memory model Overview

- **Private Memory**
 - Per work-item
- **Local Memory**
 - Shared within a work-group
- **Global /Constant Memory** (Read-only)
Visible to all work-groups
- **Host memory**
 - On the CPU



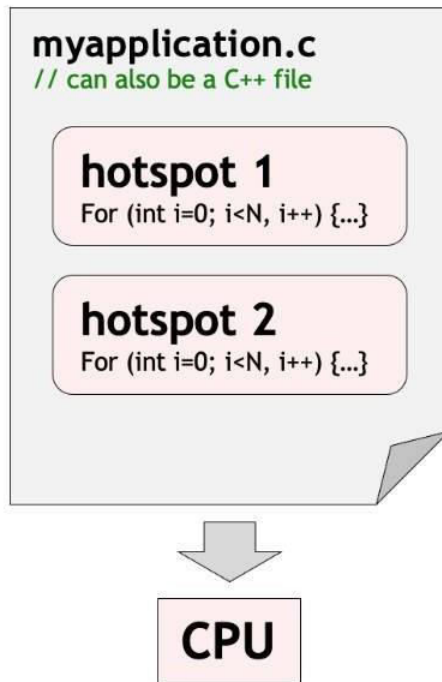
Memory management is **explicit**:

You are responsible for moving data from
host → global → local **and** back

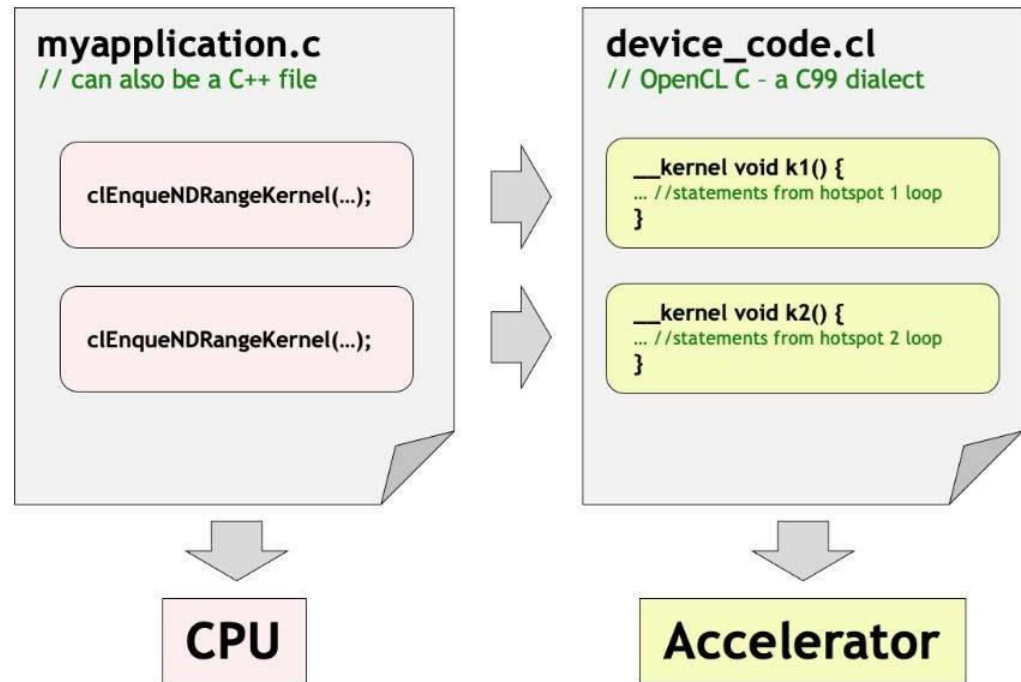


Traditional Vs. OpenCL Parallel Programming

C/C++ Programming



OpenCL Programming





The BIG idea behind OpenCL

- Replace loops with functions (a **kernel**) executing at each point in a **problem domain**
 - E.g., process a **1024x1024 image** with one kernel invocation per pixel or **1024x1024=1,048,576 kernel executions**

Traditional loops

void

```
mul(const int n,
    const float *a,
    const float *b,
    const float *c)
{
    int i;
    for (i = 0; i < n; i++)
        c[i] = a[i] * b[i];
}
```

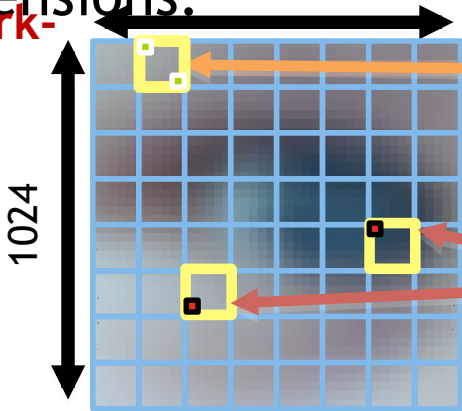
OpenCL

```
__kernel void
mul(__global const float *a,
    __global const float *b,
    __global float *c)
{
    int id = get_global_id(0);
    c[id] = a[id] * b[id];
}
// execute over n work-items
```

An N-dimensional domain of work-items

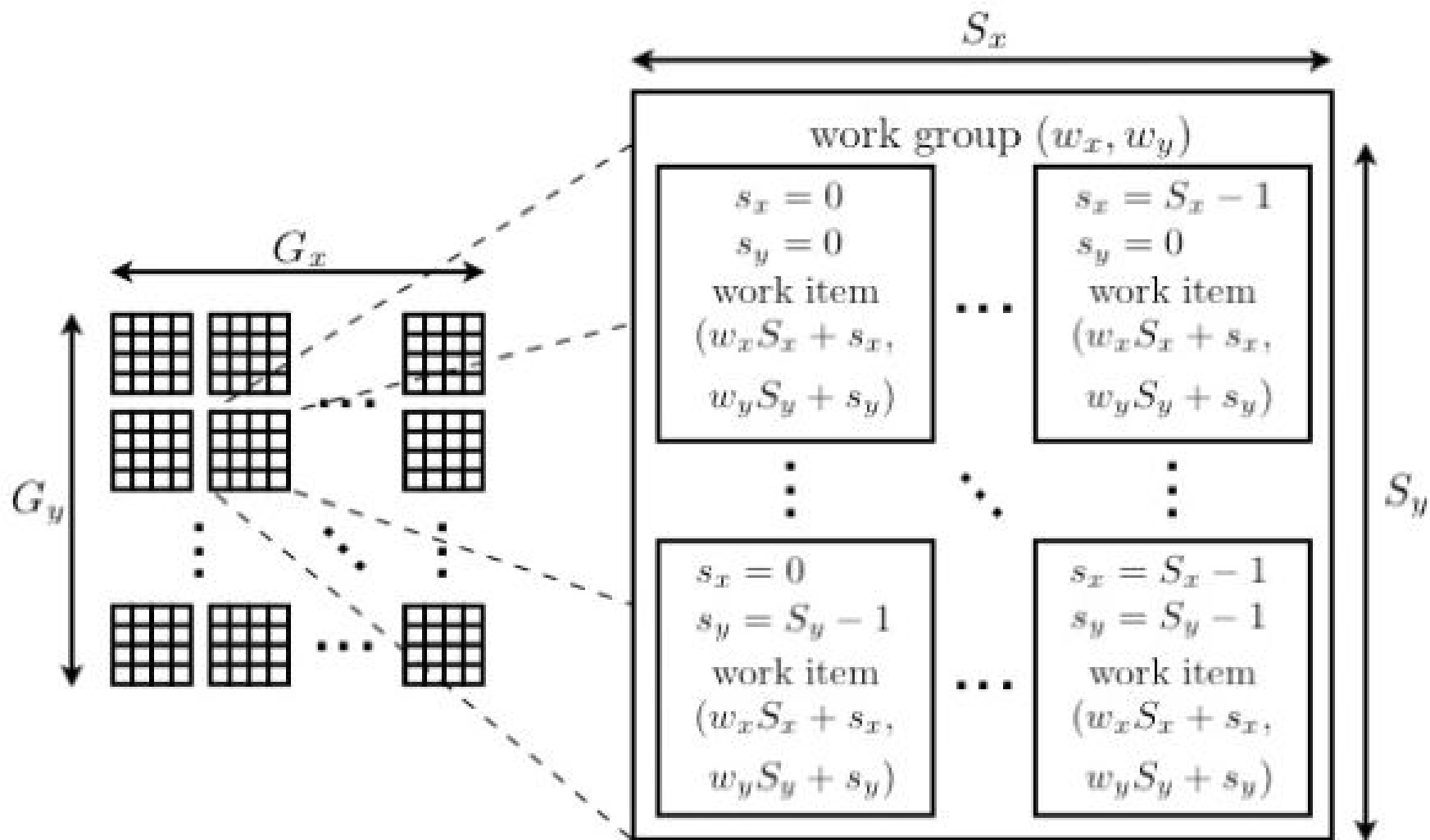
- **Global Dimensions:**
 - 1024x1024 (whole problem space)
 - For example, if we have a 2D problem with dimensions (1024, 1024), then the **NDRange** would be defined as (1024, 1024).
 - 128x128 (**work-group**, executes together)
- **Local Dimensions:**

Synchronization between items possible only within **work-groups**: **barriers** and **memory fences**

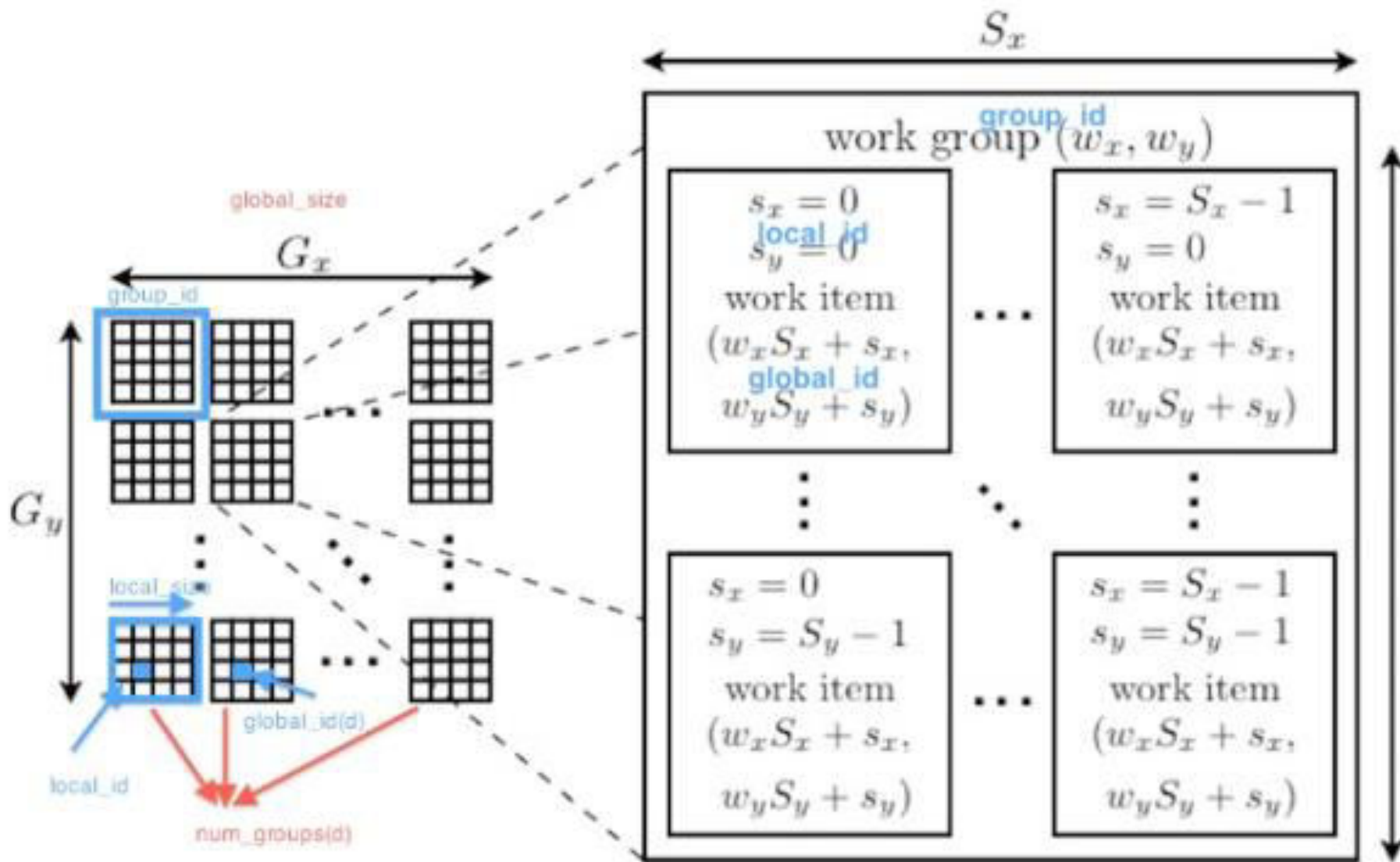


Cannot synchronize among **work-groups**
- Choose the dimensions (“best” for your algorithm):
 - 1 (one dimension)
 - 2 (two dimension)
 - 3 (three dimension)

An N-dimensional domain of work-items

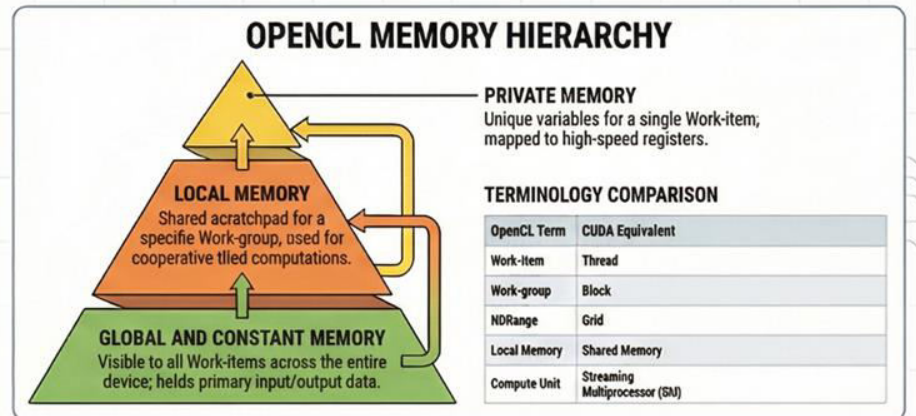
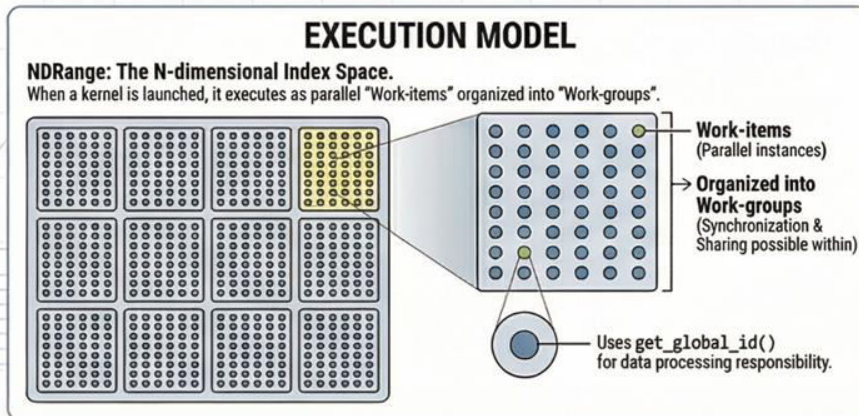
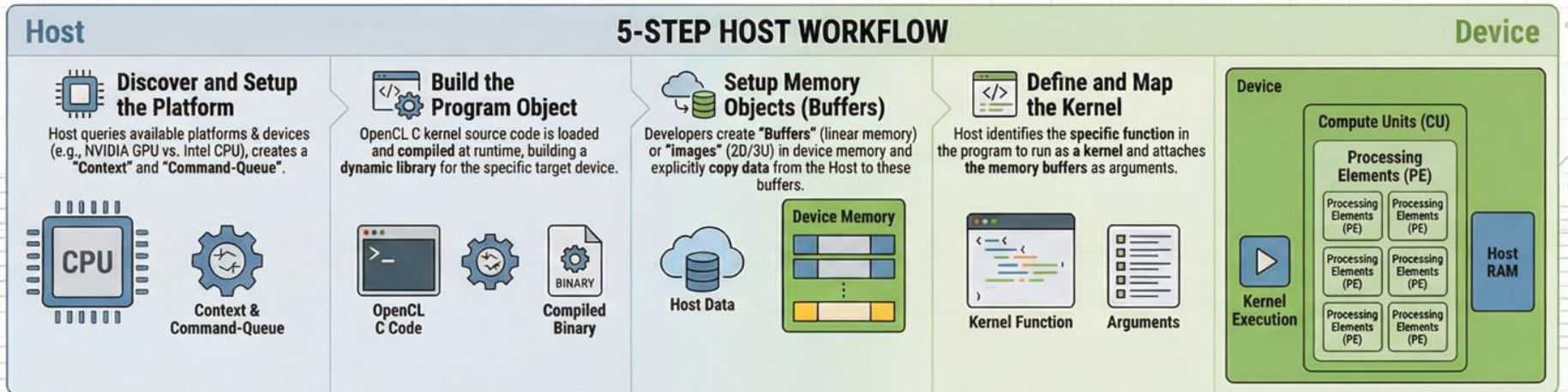


An N-dimensional domain of work-



OpenCL The OpenCL Program Workflow: Bridging the Host and Accelerator

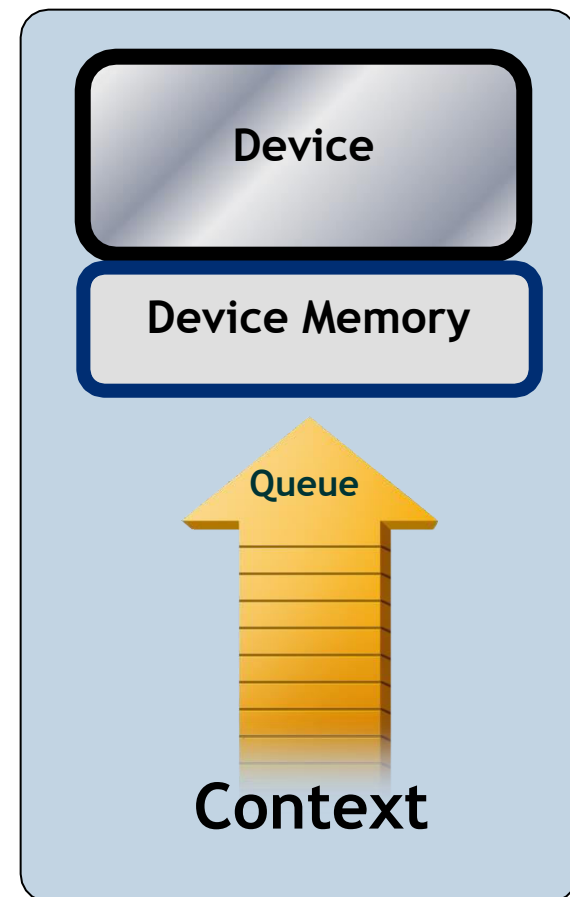
OpenCL is an open standard for cross-platform, parallel programming of heterogeneous systems, offloading compute-intensive kernels from a Host to accelerators like CPUs, GPUs, DSPs, and FPGAs.





Context and Command-Queues

- **Context:**
 - The environment within which **kernels execute** and in which **synchronization** and **memory management** is defined.
- The **context** includes:
 - **One** or **more devices**
 - **Device memory**
 - **One** or more **command-queues**
- All **commands** for a **device** (*kernel execution, synchronization, and memory operations*) are submitted through a **command-queue**.
- Each **command-queue** points to a **single device within a context**.





Building Program Objects

- The program object encapsulates:

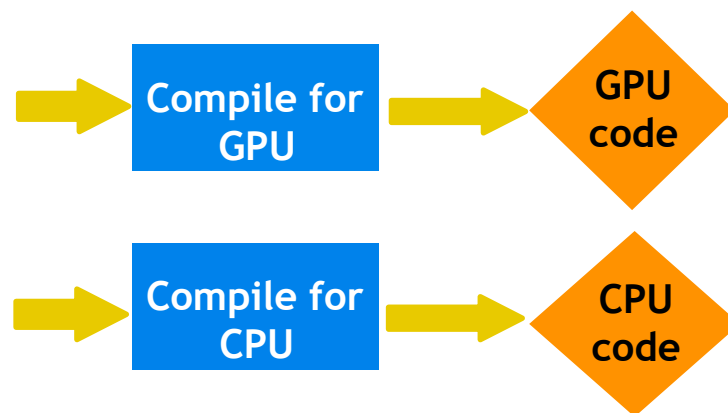
1. A **context**
2. The **program source** or **binary**, and
3. **List of target devices** and build options

- The build process to create a program object:

```
cl::Program program(context, KernelSource);
```

```
__kernel void
horizontal_reflect(write_only_image2d_t dst,
src,
{
    int x = // x-
    get_global_id(0); int y coord
    int width = // y-
    get_global_id(1);
    read_imagef(src, sampler,
(int2)(width-1-x,
y)); write_imagef(dst, (int2)(x, y),
src_val);
}
```

OpenCL uses **runtime compilation** ... because in general you don't know the details of the target device when you ship the program

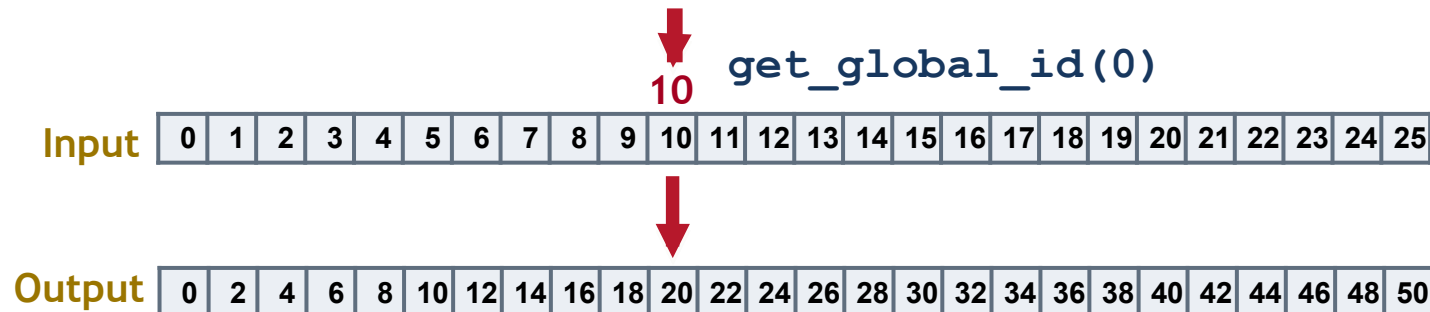




Execution model (kernels)

- **OpenCL execution model** ... define a **problem domain**
execute an **instance** of **each point** in
and a **kernel** for the
domain

```
__kernel void times_two(  
    __global float* input,  
    __global float* output)  
{  
    int i = get_global_id(0);  
    output[i] = 2.0f *  
    input[i];  
}
```





Vector Addition - Kernel

```
kernel void
vadd(
    global const float *a,
    global const float *b,
    global float *c)
{
    int gid =
    get_global_id(0);
    c[gid] = a[gid] + b[gid];
}
```



UNDERSTANDING THE HOST PROGRAM



Vector Addition – Host

- The host program is the code that runs on the host to:
 - Setup the environment for the OpenCL program
 - Create and manage kernels
- 5 simple steps in a basic host program:
 1. Define the *platform* ... platform = devices+context+queues
 2. Create and Build the *program* (dynamic library for kernels)
 3. Setup *memory* objects
 4. Define the *kernel* (attach arguments to kernel function)
 5. Submit *commands* ... transfer memory objects and execute kernels



The C++ Interface

- Khronos has defined a common C++ header file containing a high level interface to OpenCL, **cl.hpp**
- Key features:
 - **Uses common defaults** for the **platform** and **command-queue**,
 - **Simplifies** the **basic API**
 - Ability to “call” a kernel from the host, like a regular function
 - **Error checking** can be **performed with C++ exceptions**



C++ Interface: setting up the host program

- Enable OpenCL API **Exceptions**.

```
#define __CL_ENABLE_EXCEPTIONS
```

Do this **before**
including the
header files

- Include **key header files** ... both standard and custom

```
#include <CL/cl.hpp> // Khronos C++ Wrapper API  
#include <cstdio>     // C style IO (e.g. printf)  
#include <iostream>   // C++ style IO  
#include <vector>      // C++ vector types
```



1. Create a context and queue

- Grab a **context** using a **device type**:

```
cl::Context context(CL_DEVICE_TYPE_DEFAULT) ;
```

Or...
CL_DEVICE_TYPE_CPU,
CL_DEVICE_TYPE_GPU,
CL_DEVICE_TYPE_ACCELERATOR,
etc.

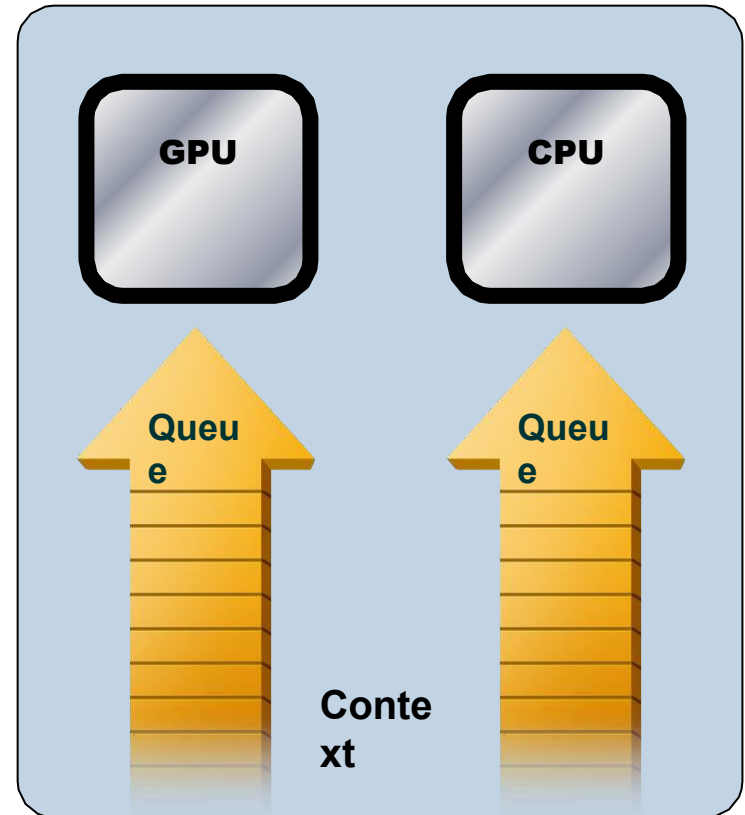
- **Create a command queue for the device in the context:**

```
cl::CommandQueue queue(context) ;
```



Commands and Command-Queues

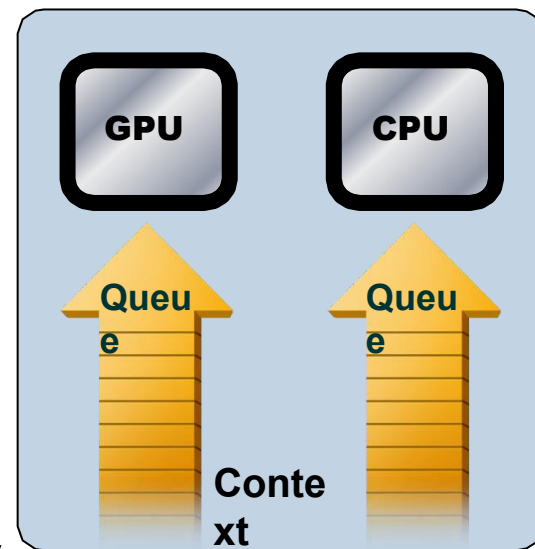
- Commands include:
 - Kernel executions
 - Memory object management
 - Synchronization
- The only way to submit **commands** to a device is through a **command-queue**.
- Each command-queue points to a **single** device within a context.





Command-Queue execution details

- *Command queues* can be configured in different ways to control how commands execute
- *In-order queues:*
 - Commands are enqueued and complete in the order they appear in the host program (program-order)
- *Out-of-order queues:*
 - Commands are enqueued in order but can execute (and complete) in any order.





2. Create and Build the

program

- Define **source code** for the **kernel-program** either as a **string literal** (for small programs) or **read it from a file** (for large applications).
- Create the **program object and compile** :

“true” tells OpenCL to build (compile/link) the program object

```
cl::Program program(context, KernelSource, true);
```

KernelSource is a **string** ... either statically set in the host program or returned from a function that loads the kernel code from a file.



3. Setup Memory Objects

- For **vector addition** we need 3 memory objects: one each for input vectors A and B, and one for the output vector C
- Create input vectors and assign values **on the host**:
for (i =

```
std::vector<float> h_a(LENGTH), h_b(LENGTH), h_c(LENGTH);  
0; i < LENGTH; i++) {  
    h_a[i] = rand() /  
    (float)RAND_MAX; h_b[i] = rand()  
    / (float)RAND_MAX;
```

- Define **OpenCL device buffers** and copy from **host buffers**:

```
cl::Buffer d_a(context, h_a.begin(), h_a.end(), true);  
cl::Buffer d_b(context, h_b.begin(), h_b.end(), true);  
cl::Buffer d_c(context, CL_MEM_WRITE_ONLY,  
  
sizeof(float)*LENGTH);
```

or CL_MEM_READ_ONLY or CL_MEM_READ_WRITE



What do we put in device memory?

- Memory Objects:
- There are two kinds of memory object
 - *Buffer* object (Always linear):
 - Defines a **linear collection of bytes**.
 - *Image* object:
 - Defines a **two- or three-dimensional region of memory**.
 - Image data can **only** be accessed with read and write functions



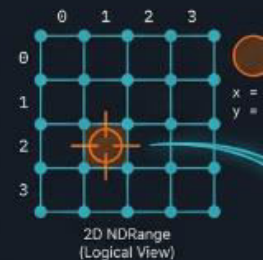
Example 1: 1D Vector Addition (Execution)

```
__kernel void vadd(  
    __global const float* A,  
    __global const float* B,  
    __global float* C,  
    const int N) {  
  
    size_t gid = get_global_id(0);  
  
    if (gid < N) {  
        C[gid] = A[gid] + B[gid];  
    }  
}
```

Boundary Guard: Prevents out-of-bounds memory access when the host program rounds global sizes up to fit perfectly into work-group multiples.

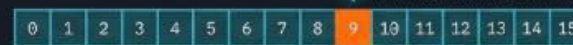
Example 2: 2D Matrix Addition (Execution)

```
__kernel void madd(__global const float* A,  
    __global const float* B,  
    __global float* C,  
    const int rows, const int cols) {  
  
    int x = get_global_id(0); // Column  
    int y = get_global_id(1); // Row  
  
    if (x < cols && y < rows) {  
        int idx = (y * cols) + x;  
        C[idx] = A[idx] + B[idx];  
    }  
}
```



2D NDRange
(Logical View)

Row-Major Memory Flattening



Flat Linear Memory
(Physical View)

$\text{int idx} = (y * \text{cols}) + x;$

The Pixel Map



Example 3: 2D Image Processing (Execution)

```

...
__kernel void process_image(read_only image2d_t src,
                           write_only image2d_t dst,
                           sampler_t sampler) {

    int x = get_global_id(0);
    int y = get_global_id(1);

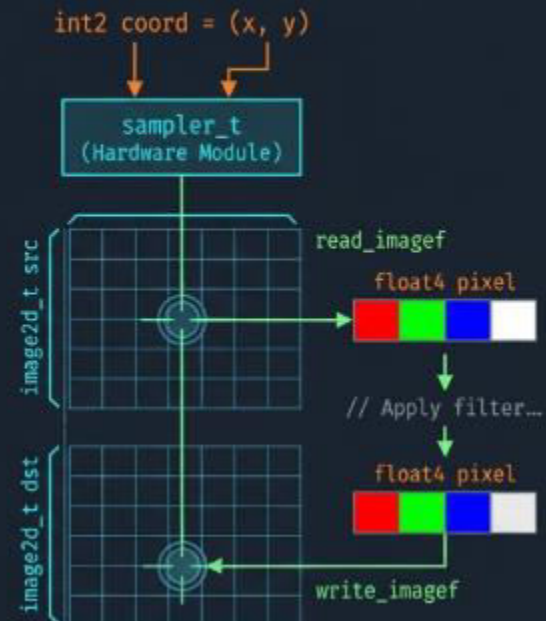
    int2 coord = (int2)(x, y);

    float4 pixel = read_imagef(src, sampler, coord);

    // Apply filter or brighten pixel here

    write_imagef(dst, coord, pixel);
}

```





Creating and manipulating

buffers

- Buffers are declared on the host as object
type: **Buffer**

- Arrays in host memory hold your original host-side data:

std::vector<float> h_a, h_b;

- Create the device-side **buffer** (d_a), assign read-only memory to hold the host array (h_a) and copy it into device memory:

```
cl::Buffer d_a(context, h_a.begin(), h_a.end(), true);
```

Start_iterator and end_iterator for the container holding host side object

Stipulates that this is a read-only buffer

or use function **clCreateBuffer ()** to create a device-side buffer



Creating and manipulating

buffers

Buffer Access Control

- The last argument sets the read/write access to the Buffer by the device. true means 'read only' while false (the default) means 'read/write'.

Data Transfer Operations

- Submit command to copy the device buffer back to host memory in array 'h_c':

```
cl::copy(queue, d_c, h_c.begin(), h_c.end());
```

Device Buffer (d_c)  Host Memory (h_c)

- Can also copy host memory to device buffers:

```
cl::copy(queue, h_c.begin(), h_c.end(), d_c);
```

Host Memory (h_c)  Device Buffer (d_c)



Creating and manipulating

```
cl_context context; // assume this is a valid OpenCL context
cl_mem buffer; // the buffer object we will create
cl_int err;

size_t size = 1024; // size of the buffer in bytes

// create a read/write buffer on the device
buffer = clCreateBuffer(context, CL_MEM_READ_WRITE, size, NULL, &err);
if (err != CL_SUCCESS) {
    // handle error
    std::cerr << "Error creating buffer: " << err << std::endl;
    return err;
}

// use the buffer as needed...

// release the buffer when done
clReleaseMemObject(buffer);
```

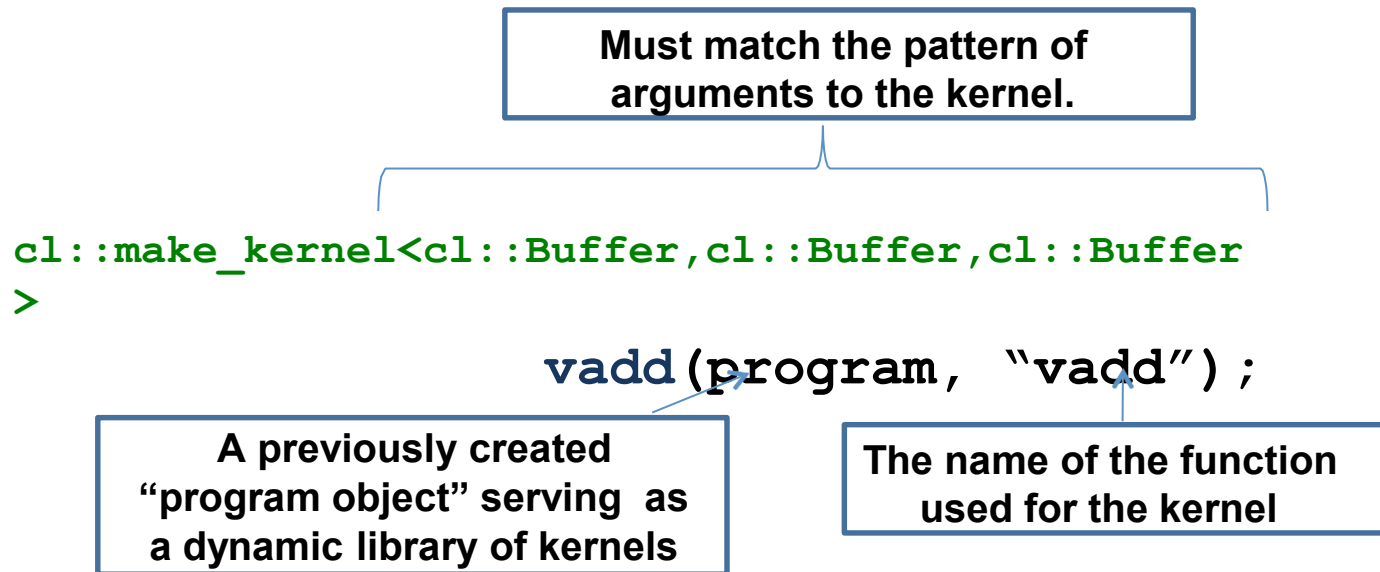
Creates a buffer object.

```
cl_mem clCreateBuffer(cl_context context,
                      cl_mem_flags flags,
                      size_t size,
                      void *host_ptr,
                      cl_int *errcode_ret)
```



4. Define the kernel

- Create a *kernel function* for the kernels you want to be able to call in the *program*:



- This means you can ‘call’ the kernel as a ‘function’ in your host code to enqueue the kernel.



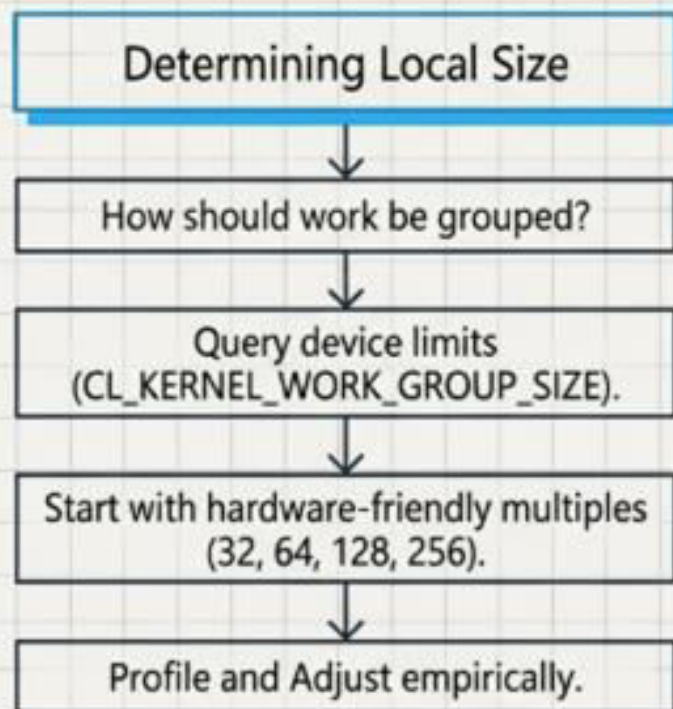
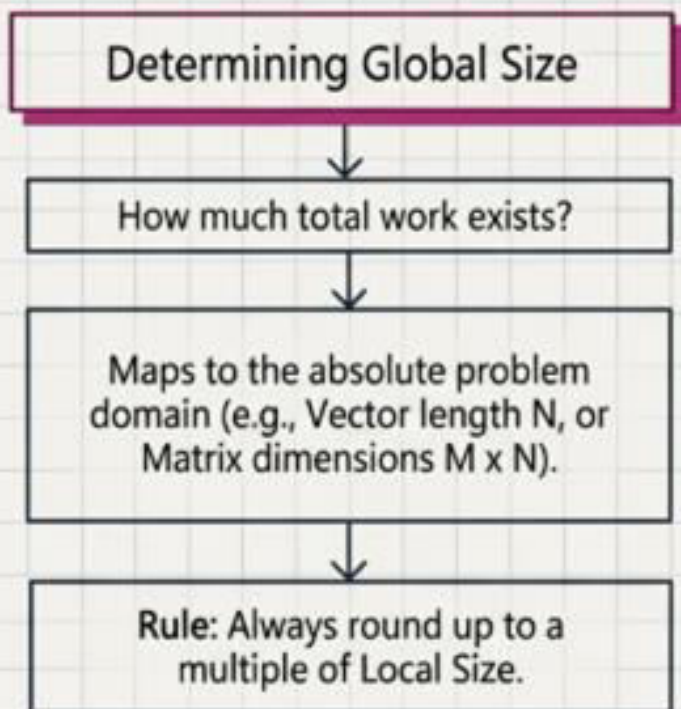
5. Enqueue commands

- For kernel launches, specify *global* and *local* dimensions

- `cl::NDRange global(1024)`
- `cl::NDRange local(64)`

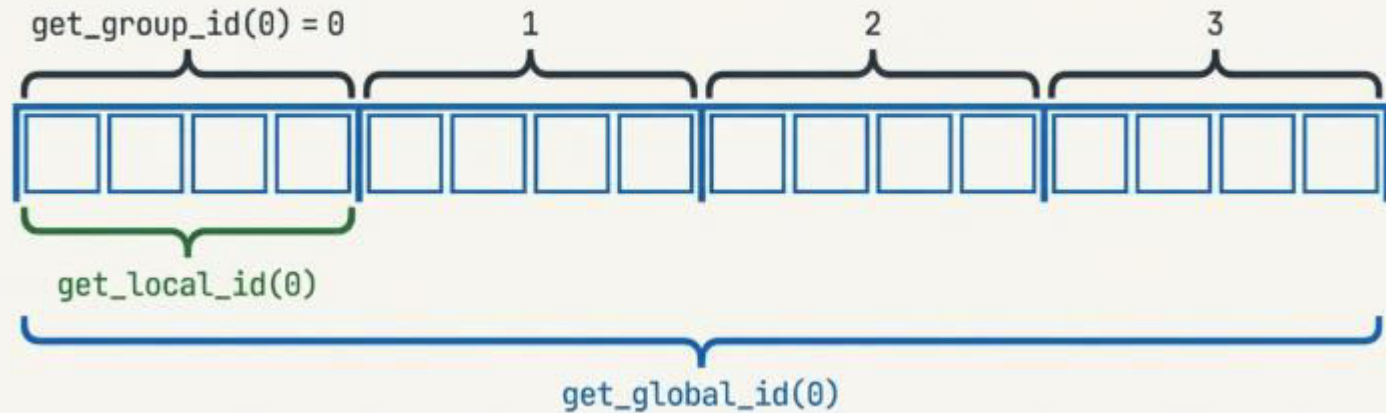
If you don't specify a local dimension, it is assumed as

Global size dictates the total volume of work. Local size dictates the physical execution grouping. While global size is mathematically fixed by your data, local size is an empirical tuning parameter.





$$\text{global_id} = \text{group_id} * \text{local_size} + \text{local_id}$$



Hardware schedules work in groups. If you specify an explicit local size, the global size must be rounded up to a multiple of that local size so all work-groups are complete.

The Math Fails

Problem size: $N = 1000$

Local size = 64

$1000 / 64 = 15.625$



The hardware cannot launch a fraction of a work-group. Launch fails.

Round Up on Host, Guard on Device

Host-side calculation (C++)

```
size_t globalSize =  
    ((N + localSize - 1) / localSize) *  
    localSize
```

(Evaluates to 1024)

Device-side guard (OpenCL)

```
if (gid < (size_t)N) {  
    // Kernel Logic  
}
```

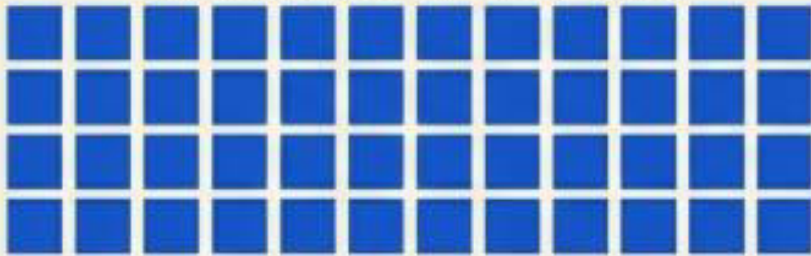
The 24 extra work-items safely exit early.



The Portability Paradox

OpenCL code is portable across CPUs, GPUs, and accelerators. But because devices are architecturally fundamentally different, an optimized local size on one device may cripple performance on another.

Profile A (GPU Accelerator)



Compute Units: 24
Max Group Size: 256
Preferred Multiple: 32

Profile B (CPU Runtime)



Compute Units: 4
Max Group Size: 8192
Preferred Multiple: 1

Portable code does not imply portable performance.
Optimization must be informed by runtime device queries.



5. Enqueue commands

Enqueue the kernel for execution (note: returns immediately ... i.e. this is a **non-blocking command**):

```
vadd(c1::EnqueueArgs(queue, global), d_a, d_b, d_c);
```

Read back result (as a **blocking operation**). We use an in-order queue to assure the previous commands are completed before the read can begin

```
c1::copy(queue, d_c, h_c.begin(), h_c.end());
```



OpenCL Vector Addition

```
#include <iostream>
#include <vector>
#include <CL/cl.hpp>

int main() {
    std::vector<float> a = {1.0f, 2.0f, 3.0f, 4.0f};
    std::vector<float> b = {4.0f, 3.0f, 2.0f, 1.0f};
    std::vector<float> c(a.size());

    try {
        // get available OpenCL platforms
        std::vector<cl::Platform> platforms;
        cl::Platform::get(&platforms);

        // choose a platform
        cl::Platform platform = platforms[0];

        // get available OpenCL devices
        std::vector<cl::Device> devices;
        platform.getDevices(CL_DEVICE_TYPE_ALL, &devices);

        // choose a device
        cl::Device device = devices[0];

        // create an OpenCL context for the device
        cl::Context context({device});

        // create an OpenCL program from source
        cl::Program program(context, "kernel.cl");
```



OpenCL Vector Addition

Example

```
// build the program for the device
program.build({device});
```

```
// create OpenCL buffers for the input and output vectors
```

```
cl::Buffer bufA(context, CL_MEM_READ_ONLY, a.size() * sizeof(float));
```

```
cl::Buffer bufB(context, CL_MEM_READ_ONLY, b.size() * sizeof(float));
```

```
cl::Buffer bufC(context, CL_MEM_WRITE_ONLY, c.size() * sizeof(float));
```

```
// create a command queue for the device
```

```
cl::CommandQueue queue(context, device);
```

```
// enqueue data to be transferred to the device
```

```
queue.enqueueWriteBuffer(bufA, CL_TRUE, 0, a.size() * sizeof(float), a.data());
```

```
queue.enqueueWriteBuffer(bufB, CL_TRUE, 0, b.size() * sizeof(float), b.data());
```

```
// create a kernel object for the vector addition kernel
```

```
cl::Kernel kernel(program, "vecadd");
```

```
// set the arguments of the kernel
```

```
kernel.setArg(0, bufA);
```

```
kernel.setArg(1, bufB);
```

```
kernel.setArg(2, bufC);
```

```
// enqueue the kernel for execution
```

```
queue.enqueueNDRangeKernel(kernel, cl::NullRange, cl::NDRange(a.size()), cl::NullRange);
```

```
// enqueue data to be transferred back to the host
```

```
queue.enqueueReadBuffer(bufC, CL_TRUE, 0, c.size() * sizeof(float), c.data());
```

True:
Readonly

Global offset

Global NDRange

LocalNDRange



OpenCL Vector Addition Example

```
// print the result
```

```
for (float x : c) {  
    std::cout << x << " ";  
}
```

```
std::cout << std::endl;
```

```
} catch (cl::Error& e) {
```

```
    std::cerr << "OpenCL error: " << e.what() << " (" << e.err() << ")" <<
```

```
std::endl;
```

```
    return 1;
```

```
}
```

```
return 0;
```

```
}
```



INTRODUCTION TO OPENCL KERNEL PROGRAMMING



OpenCL kernel

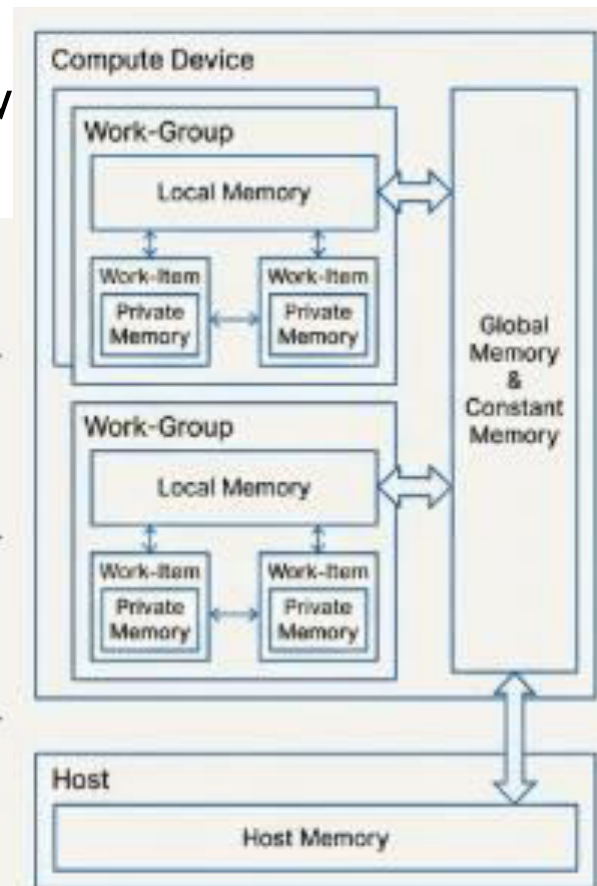
- **Derived from ISO C99**
 - A few *restrictions*: no recursion, function pointers, functions in C99 standard headers ...
 - Preprocessing directives defined by C99 are supported (#include etc.)
- **Built-in data types**
 - **Scalar** and **vector data types**, **pointers**
 - **Data-type conversion functions**:
 - `convert_type<_sat><_roundingmode>`
 - **Image types**: `image2d_t`, `image3d_t` and `sampler_t`



OpenCL C Language Highlights

- **Function qualifiers**
 - `__kernel` qualifier **declares** a **function** as a **kernel**
 - I.e. makes it visible to host code so it can be enqueued
- **Address space qualifiers**
 - `__global`, `__local`, `__constant`,
 - Pointer kernel arguments must be declared with an **address space qualifier**

Scope	Syntax	Speed	Accessibility	Typical Use Case
Private	<code>float temp;</code>	Extremely Fast (Registers)	One work-item only	Intermediate math, loop counters
Local	<code>__local float tile[];</code>	Fast (On-chip Shared)	All items in one group	Tiled matrix multiplication, stencils
Global	<code>__global float* A;</code>	Slow (DRAM)	All work-groups & Host	Large input/output arrays





OpenCL C Language Highlights

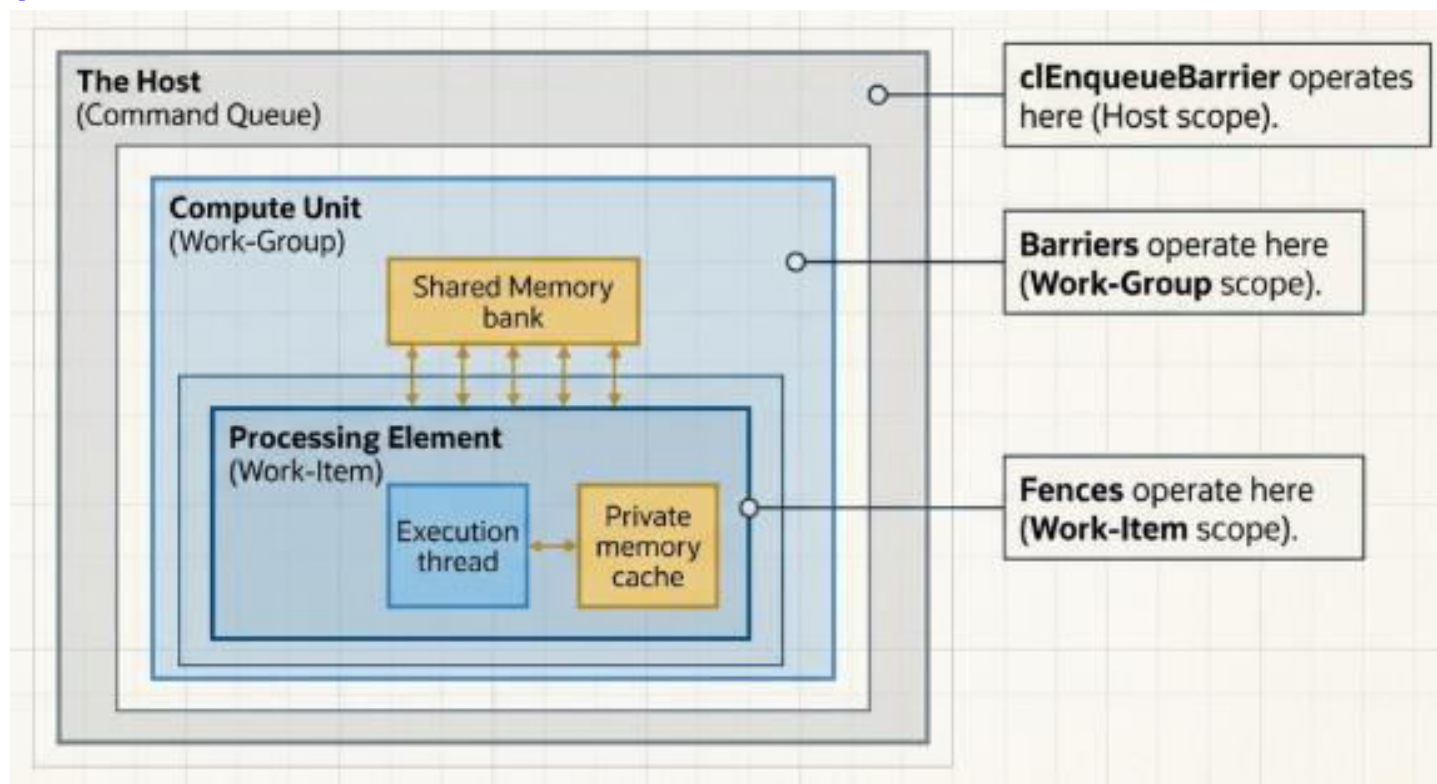
- `uint get_work_dim()` ... number of dimensions in use (1,2, or 3)
- `size_t get_global_id(uint n)` ... global work-item ID in dim “n”
- `size_t get_local_id(uint n)` ... work-item ID in dim “n” inside work-group
- `size_t get_group_id(uint n)` ... ID of work-group in dim “n”
- `size_t get_global_size(uint n)` ... num of work-items in dim “n”
- `size_t get_local_size(uint n)` ... num of work-items in work group in dim “n”



OpenCL C Language Highlights

- **Synchronization functions**

- **Barriers** - **all work-items** within a **work-group** must **execute the barrier function** before **any work-item can continue**
- **Memory fences** - provides **ordering between memory operations**





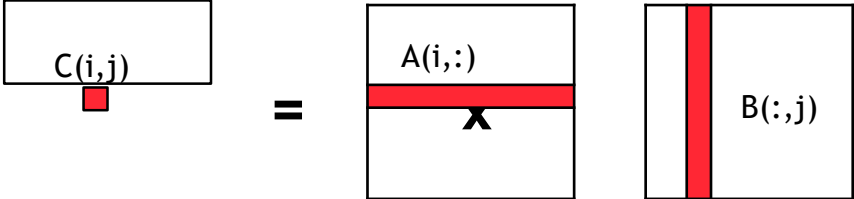
- **Pointers to functions** are *not allowed*
- **Pointers to pointers** allowed *within* a kernel,
but not as an argument to a kernel invocation
- **Variable length arrays and structures** are
not supported
- **Recursion** is not supported (yet!)



Matrix multiplication: sequential code

We calculate $C=AB$, $\dim A = (N \times P)$, $\dim B=(P \times M)$, $\dim C=(N \times M)$

```
void mat_mul(int Mdim, int Ndim, int Pdim,
             float *A, float *B, float *C)
{
    int i, j, k;
    for (i = 0; i < Ndim; i++) {
        for(j = 0; j < Mdim; j++) {
            C[i*N+j] = 0.0f;
            for (k = 0; k < Pdim; k++) {
                // C(i, j) = sum(over k) A(i,k) * B(k,j)
                C[i*Ndim+j] += A[i*Ndim+k] *
                               B[k*Pdim+j];
            }
        }
    }
}
```



Dot product of a row of A and a column of B for each element of C

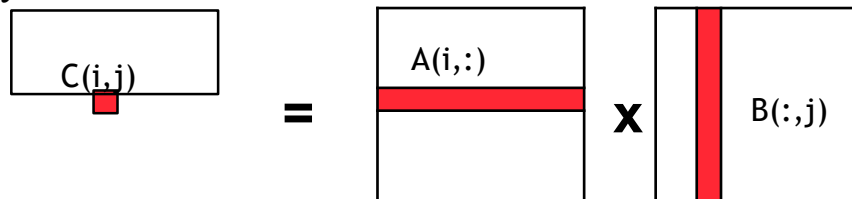


Matrix multiplication: sequential

We calculate $C=AB$, where all three matrices are $N \times N$

```
void mat_mul(int N, float *A, float *B, float *C)
{
    int i, j, k;
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            C[i*N+j] = 0.0f;
            for (k = 0; k < N; k++) {
                // C(i, j) = sum(over k) A(i,k) *
                B(k,j)  C[i*N+j] += A[i*N+k] *
                B[k*N+j];
            }
        }
    }
}
```

Let's make it
easier and
specialize to
square matrices



Dot product of a row of A and a column of B for each element of C



Matrix multiplication performance

- Serial C code on CPU (single core).

Case	MFLOPS	
	CPU	GPU
Sequential C (not OpenCL)	887.2	N/A

Device is Intel® Xeon® CPU, E5649 @ 2.53GHz
using the gcc compiler.

These are not official benchmark results. You may observe completely different results should you run these tests on your own system.

Third party names are the property of their owners.



Matrix multiplication: sequential code

```
void mat_mul(int N, float *A, float *B, float *C)
{
    int i, j, k;
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            C[i*N+j] = 0.0f;
            for (k = 0; k < N; k++) {
                // C(i, j) = sum(over k) A(i,k) * B(k,j)
                C[i*N+j] += A[i*N+k] *
                    B[k*N+j];
            }
        }
    }
}
```



Matrix multiplication: sequential code

```
void mat_mul(int N, float *A, float *B, float *C)

{
    int i, j, k;
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            C[i*N+j] = 0.0f;
            for (k = 0; k < N; k++) {
                // C(i, j) = sum(over k) A(i,k) *
                B(k,j)  C[i*N+j] += A[i*N+k] *
                B[k*N+j];
            }
        }
    }
}
```



Matrix multiplication: OpenCL kernel (1/2)

```
__kernel void mat_mul(const int N, __global float *A,
                      __global float *B, __global float
{
    int i, j, k;
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            C[i*N+j] = 0.0f;
            for (k = 0; k < N; k++) {
                // C(i, j) = sum(over k) A(i,k) * B(k,j)
                C[i*N+j] += A[i*N+k] * B[k*N+j];
            }
        }
    }
}
```

Mark as a kernel function and
specify memory qualifiers



Matrix multiplication: OpenCL kernel (2/2)

```
__kernel void mat_mul(const int N, __global float *A,
                      __global float *B, __global float
{
    int i, j, k;
    i = get_global_id(0);
    j = get_global_id(1);
    C[i*N+j] = 0.0f;
    for (k = 0; k < N; k++) {
        // C(i, j) = sum(over k) A(i,k) *
        B(k,j)  C[i*N+j] += A[i*N+k] *
        B[k*N+j];
    }
}
}
```

Replace loops with the
work item's global id



Matrix multiplication: kernel cleaned-up

Rearrange and use a local scalar for intermediate C element values (a common optimization in Matrix Multiplication functions)

```
{
```

```
__kernel void mmul(  
    const int N,  
    __global float *A,  
    __global float *B,  
    __global float *C)  
{  
    int k;  
    int i = get_global_id(0);  
    int j = get_global_id(1);  
    float tmp = 0.0f;  
    for (k = 0; k < N; k++)  
        tmp += A[i*N+k]*B[k*N+j];  
  
    C[i*N+j] = tmp;  
}
```



Matrix multiplication performance

- Matrices are stored in global memory.

Case	MFLOPS	
	CPU	GPU
Sequential C (not OpenCL)	887.2	N/A
C(i,j) per work-item, all global	3,926. 1	3,720.9

Device is Tesla® M2090 GPU from NVIDIA® with a max of 16 compute units, 512 PEs

Device is Intel® Xeon® CPU, E5649 @ 2.53GHz

These are not official benchmark results. You may observe completely different results should you run these tests on your own system.

Third party names are the property of their owners.



Another Test & Optimization

Level 2: Naïve 2D Matrix Multiplication

Applying the direct mapping strategy to a 2D problem: one work-item computes one cell of the product matrix C. The result? The GPU performs worse than a standard CPU. A conceptually correct kernel does not guarantee a performant one.

```
int i = get_global_id(0);
int j = get_global_id(1);
float tmp = 0.0;
for(int k=0; k<Pdim; k++) {
    tmp += A[i*Ndim+k] * B[k*Pdim+j];
}
C[i*Ndim+j] = tmp;
```

CPU Sequential

744 MFLOPS

GPU Naïve

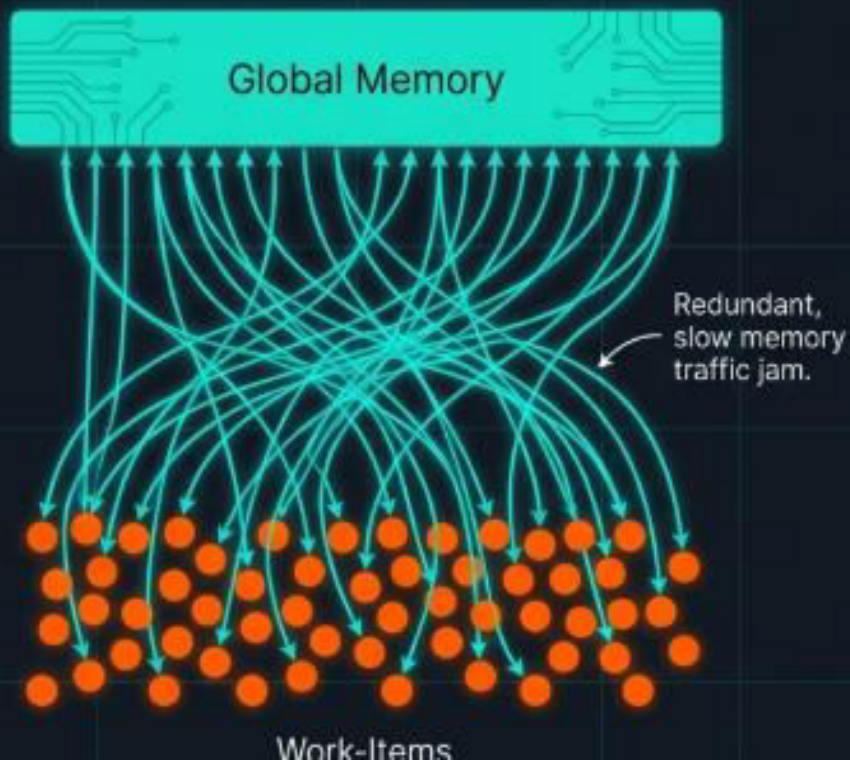
511 MFLOPS

← Bottleneck Detected:
Memory Bound

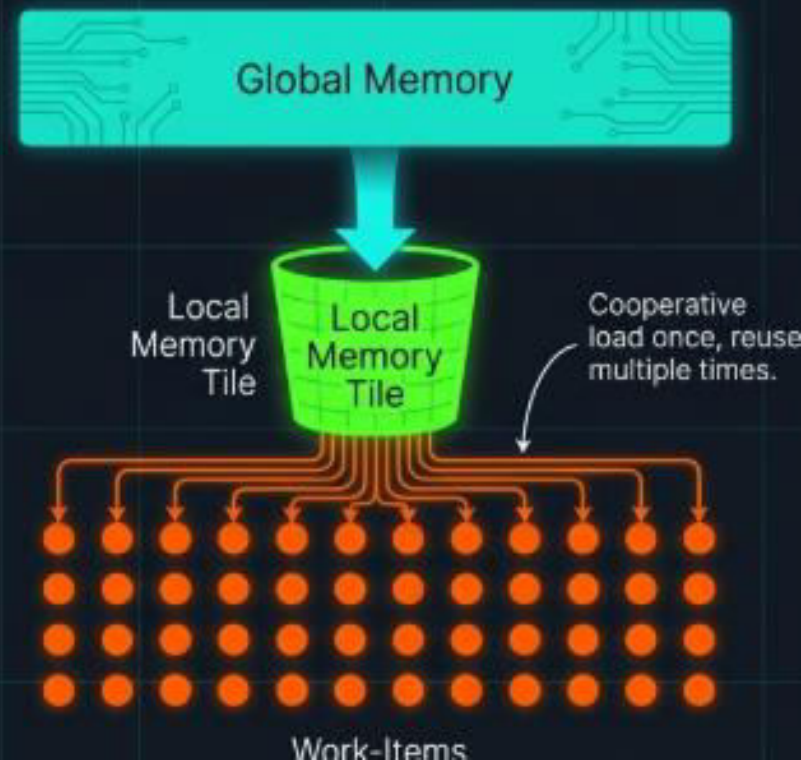
Advanced Design: The Local Memory Advantage

Problem: Naive processing forces work-items to repeatedly fetch overlapping data from slow Global Memory.

Naive Pattern: High Latency



Tiled Pattern: High Throughput

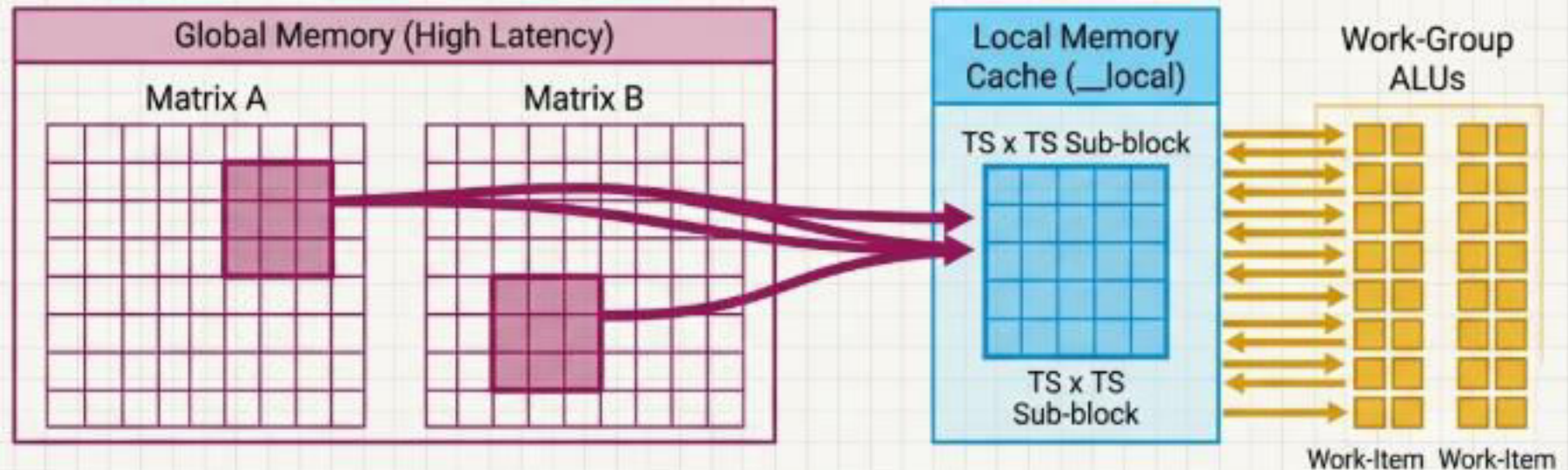




$$C[row][col] = \sum_{k=0}^{K-1} A[row][k] \cdot B[k][col]$$

The Solution: The Tiling Engine

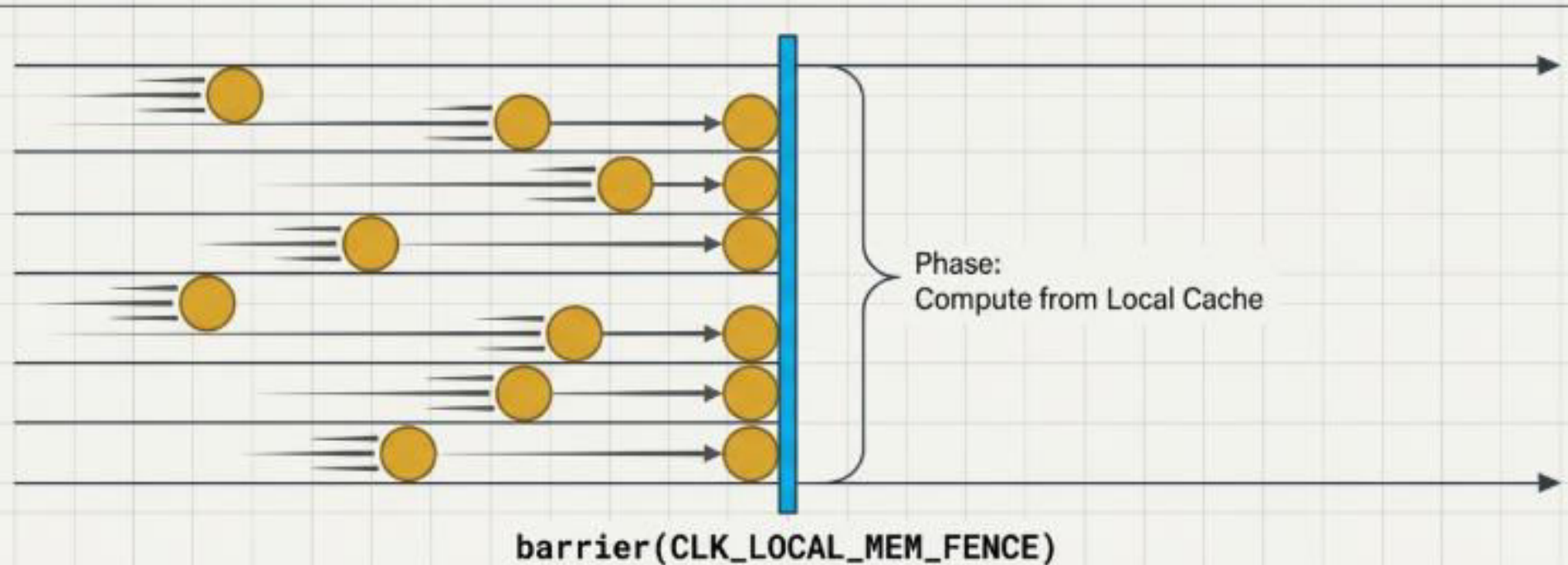
Instead of work-items fetching data independently, the Work-Group acts as a **cooperative team**. They execute a coordinated fetch of a matrix tile from Global Memory into fast Local Memory. The entire group then reuses that locally cached data, drastically slashing global bandwidth demands.





Enforcing Cooperation: The Barrier Wall

Because work-items share local memory, strict synchronization is mandatory to prevent race conditions. Two barriers are required per loop iteration: Wait to Compute (ensures the tile is fully loaded) and Wait to Overwrite (ensures every item is finished computing).





Level 3: Tiled Matrix Multiplication

By allocating `__local float tileA[TS][TS]`, the kernel establishes a user-managed cache. The cooperative loop operates in three chronologically distinct phases: fetch, synchronize, and compute.

Step 1: Fetch

```
tileA[lr][lc] = A[row*K + aCol];  
tileB[lr][lc] = B[bRow*N + col];
```

Work-items independently
load one element into shared
Local Cache.

Step 2: Synchronize

```
barrier(CLK_LOCAL_MEM_FENCE);
```

Execution halts. Prevents
read-before-write race
conditions.

Step 3: Compute

```
for(int k=0; k<TS; k++) {  
    sum += tileA[lr][k] * tileB[k][lc];  
}
```

ALUs rapidly process dot
products directly from
high-speed cache.

Level 3: Tiled Matrix Multiplication

```
#define TS 16

__kernel void matmul_tiled(__global const float* A, __global const float* B,
                          __global float* C, const int M, const int N, const int K)
{
    int row = get_global_id(0);
    int col = get_global_id(1);
    int lr = get_local_id(0);
    int lc = get_local_id(1);

    __local float tileA[TS][TS];
    __local float tileB[TS][TS];
    float sum = 0.0f;
    for (int t = 0; t < (K + TS - 1) / TS; t++) {
        int aCol = t * TS + lc;
        int bRow = t * TS + lr;
        if (row < M && aCol < K) tileA[lr][lc] = A[row * K + aCol];
        else tileA[lr][lc] = 0.0f;

        if (bRow < K && col < N) tileB[lr][lc] = B[bRow * N + col];
        else tileB[lr][lc] = 0.0f;

        barrier(CLK_LOCAL_MEM_FENCE);
        for (int k = 0; k < TS; k++) {
            sum += tileA[lr][k] * tileB[k][lc];
        }
        barrier(CLK_LOCAL_MEM_FENCE);
    }
    if (row < M && col < N)
        C[row * N + col] = sum;
}
```



The Value of Explicit Architecture

OpenCL does not guarantee performance through syntax alone. By adapting the algorithmic design to respect the physical memory hierarchy of the hardware, the tiled approach completely eliminates the memory bottleneck, yielding nearly a 5x performance leap over the naive implementation.

CPU Sequential

Scope: Single Thread
(Optimized ikj loop)

Memory Pattern: Relies on
automatic L1/L2 Cache

Benchmark Speed:
1130 MFLOPS

GPU Naïve

Scope:
1 Work-Item per Cell

Memory Pattern:
100% Global Memory Fetches

Benchmark Speed:
511 MFLOPS

[Memory Bound]

GPU Tiled

Scope:
Work-Group Cooperation

Memory Pattern:
Explicit Local Memory Caching

Benchmark Speed:
2472 MFLOPS

[Compute Bound]



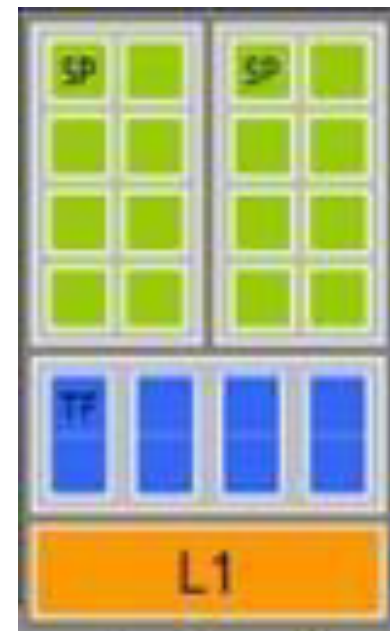
OpenCL and CUDA

- Many OpenCL features have a one to one mapping to CUDA features
- **OpenCL**
 - More complex platform and device management
 - More complex kernel launch
 - More lower level than CUDA



OpenCL and CUDA

- **Compute Unit (CU)** correspond to
 - CUDA streaming multiprocessors (SMs)
 - CPU core
 - etc.
- **Processing Element** correspond to
 - CUDA streaming processor (SP)
 - CPU ALU





OpenCL and CUDA

- *Work Item* (CUDA *thread*) – executes kernel code
- *Index Space* (CUDA *grid*) – defines work items and how data is mapped to them
- *Work Group* (CUDA *block*) – work items in a work group can synchronize



References

Optimizing OpenCL applications on Intel Xeon Phi

<http://iwocl.org/wp-content/uploads/2013/06/Optimizing-OpenCL-Applications-on-Intel-Xeon-Phi-IWOCL.pdf>

OpenCL home page

<http://www.khronos.org/opencl/>

One OpenCL to Rule Them All

Romain Dolbeau, Francois Bodin, Guillaume Colin de Verdière

<http://www.caps-entreprise.com/wp-content/uploads/2012/08/One-OpenCL-to-rule-them-all.pdf>

Ramses Project

Romain Teissier, Pierre-François Lavallée, et al.

http://irfu.cea.fr/Phoce/Vie_des_labos/Ast/ast_sstechnique.php?id_ast=904